

Generalized DBLog: A Verified Contract for Interleaving Copied Rows with a Change Log

Andreas Andreakis

Abstract

Change-data capture (CDC) feeds downstream systems like caches, search indexes, and data warehouses from a database’s log of committed row changes. When bootstrapping, adding a table, or repairing downstream data, a pipeline must also copy existing rows. Merging this copy with the active log introduces the copy-to-log handoff problem. Changes must not fall through a gap, and older copied state must not overwrite a newer logged update or resurrect a deleted row. DBLog, developed at Netflix, addressed this problem by reading tables in chunks and interleaving those reads with the live log. Watermarks identify the changes that overlap each read, and the log wins when a copied row is stale [1,2]. Debezium and Flink CDC have since adapted this design [3,4,5].

Earlier work proved that applying the original algorithm’s copied rows and logged changes in their emitted order reconstructs the source’s rows, including the effect of every logged insert, update, and delete processed [6]. Generalized DBLog asks when the same result holds for variants of that design. We state the conditions the source and capture implementation must satisfy. Once copying and reconciliation are complete, we prove that the result holds across all selected tables and key ranges even when their rows were read at different times. A single database snapshot is not required for the copy. Further logged changes advance the reconstructed state one event at a time.

We establish these guarantees for classic watermarking, Debezium’s signal-table and read-only modes, Flink CDC’s parallel chunks, reads and dumps tied to exact log positions, and engine-consistent backups whose log position lies within known bounds. The complete theory is machine-checked in Isabelle/HOL, its core independently verified in Lean 4, and the protocols are also examined by bounded model checking in TLA⁺.

Keywords

databases, database replication, change-data-capture, CDC, incremental snapshots, formal verification, Isabelle/HOL, Lean, TLA⁺

1 Introduction

1.1 The production problem

A change-data-capture (CDC) pipeline typically needs both the latest state of all rows and an ordered stream of live changes. Because log retention is finite, the log alone may not supply the needed baseline. A pipeline must therefore combine a copy of the current source state with ongoing log consumption.

This need is not limited to an initial bootstrap. An additional sink, a newly added table, or downstream data loss can require another copy later on. If only a few keys are damaged, repeating a table-wide or even instance-wide copy is unnecessary work. These recurring repair and expansion cases make a one-time bootstrap an incomplete answer for many deployments. Furthermore, the design problem is not simply how to read current rows. It is how to combine that read with the change log without losing the effect of an update or delete, or allowing older copied state to supersede newer logged state.

1.2 How copied state joins the change log

Even a copy obtained from one database snapshot does not, by itself, identify where a consumer should continue in the change log. If consumption resumes too late, changes are lost. If it resumes too early and the overlap is handled in the wrong order, an older copied row can arrive after a newer logged change and reverse it. A delete that races the copy can likewise be missed, leaving a row downstream that no longer exists at the source.

Copying state and handoff correctness are separate problems. A single snapshot still needs a valid continuation point. A chunked read can still be correct when its overlap with the log is properly delimited and reconciled. The mechanisms analyzed here reduce to two handoff shapes.

Establish an exact handoff point. Some databases can associate a read with a position in their change log. The position states which changes the read already contains, so the consumer knows where log processing should continue. A system can also establish the same proof-relevant ordering through explicit phase coordination that places the copy before the continuing suffix. A table read or an entire dump can

then be placed at that position without reconciling an interval of uncertain changes. This is a strong primitive, but the result depends on the database providing the promised relationship between the read and the recorded position [7,8,9,10]. If only an interval containing the true read position is known, the copy must then instead be reconciled with the changes in that interval.

Bracket and reconcile an overlap. A system can keep the change stream moving while it reads current state. It records positions around the read, treats the interval between them as an overlap window, and reconciles the copy with the changes in that window so that the latest logged change wins over a stale copied row. For example, suppose a read returns key 42 with value V1 and the log then records an update to V2 before the window closes. Reconciliation keeps V2. A key with no event in the window keeps its copied value, while an in-window update or delete drops it.

These approaches are coordination patterns, not mutually exclusive categories. A whole-table or whole-instance dump can use an exact position when one is available. If its position is known only to lie inside an interval, the same dump becomes one large overlapping read whose racing changes must be reconciled.

An exact handoff relies on its read-coordinate or ordering guarantee. An overlapping handoff relies on retention, complete observation, and reconciliation. Scope, repeatability, resumability, unit size, and parallelism remain separate operational choices. Operators must also evaluate source load, copy time, log lag, and recovery behavior in their own environment.

1.3 From DBLog to Generalized DBLog

DBLog uses the overlapping approach and makes the overlap small and repeatable. The original algorithm reads tables in chunks and brackets each read with two watermark events. Watermarks are changes that are applied on the source and observed in the change log afterwards. Changes inside the bracket take precedence over the corresponding copied rows, so the log wins whenever it races the copy [1,2]. The watermarks enclose the unknown read position and make each chunk’s overlap explicit. Figure 1 illustrates one such bracket. This lets handoff state and progress be managed one unit at a time. A capture can target all tables, one table, or selected keys, pause and resume after a failure, and run again later for repair while change events continue to flow [1].

Earlier work proved that replay of this original watermarked algorithm matches the source at a certain log position, representing all changes up to that point [6]. Open-source CDC projects have since adopted and adapted the design. Debezium’s documented default incremental-snapshot

Operational impact of watermark writes. Writing to the source can initially seem like an unattractive property of a CDC design. Controlled source-side writes are, however, already an established observability technique. For example, Debezium can execute a configured query that writes a heartbeat table when captured data is quiet, and Oracle GoldenGate updates source heartbeat tables and carries those records through the replication path to measure end-to-end lag [11,12]. A deployment that accepts source-side heartbeat writes has therefore already accepted the basic operational capability that DBLog watermarks require, namely small, controlled writes to a dedicated table whose events are expected to pass through the capture pipeline. Heartbeats use that capability to provide a liveness signal. DBLog watermarks reuse it to delimit chunk reads.

mode writes opening and closing window records to a configured signaling data collection under its `insert_insert` strategy. Its read-only implementation for MySQL and MariaDB instead samples server GTID state without writing those markers [4,13,14]. Flink CDC’s documented MySQL protocol brackets chunks in parallel and reconciles each chunk with the changes that raced its read. Its documentation records that the design is inspired by DBLog [5]. We also study log-coordinate-bound reads and native dumps, which realize the exact-position or uncertain-interval approaches described above [15].

These variations motivate the paper’s central question. Under what conditions can copied state be merged with the continuing log so that replay neither loses source changes nor lets older copied state overwrite newer logged state? Generalized DBLog provides a framework for answering that question. Under this framework, replaying the combined stream reconstructs the source state at a single log position, representing the source’s state history up to that point. We call this a *virtual cut*. Importantly, this cut is not stored inside DBLog. Instead, it is the state represented by replaying DBLog’s output. Unlike a one-time physical snapshot that represents all rows at one common source coordinate, a virtual cut is assembled iteratively and on an ongoing basis by interleaving chunk reads with log changes.

The following sections establish what each capture protocol must guarantee to achieve this result.

1.4 Contributions and reading paths

Contributions. We make four contributions:

- (1) We state a correctness contract for combining copied database state with a committed change log and prove its cut theorem. Replaying the combined stream matches the source for the keys processed so far (Lemma 4.3), and covers all target tables and keys once all chunks finish (Theorem 1). The correctness

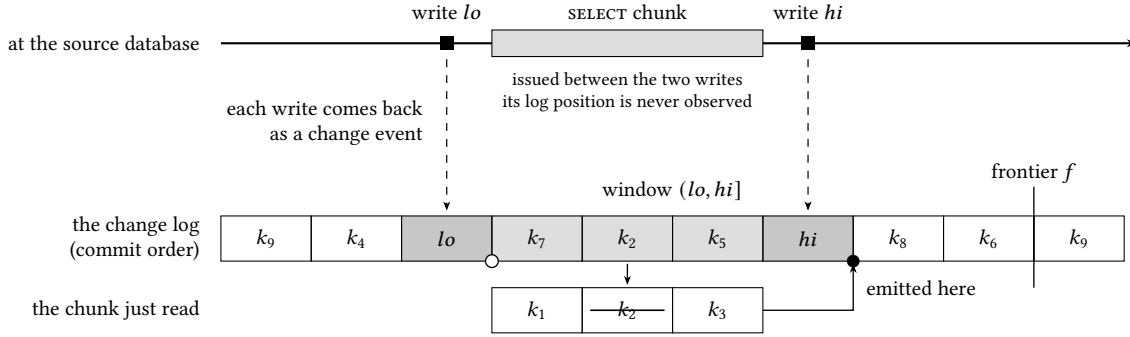


Figure 1: DBLog’s windowing. Each watermark is an update to a dedicated single-row table at the source, so it comes back from the log as an ordinary change event. Those two events are the window’s edges. The window is half-open, with the lo cell lying outside it and the hi cell inside. The chunk read is issued between the two writes, so its own log position, which is never observed, must lie inside the window. A copied row is drawn in the column of the logged change that shares its key, not at a log position of its own. The logged k_2 has already gone out, so the copied k_2 is discarded, and only the survivors are emitted when hi arrives.

condition is per key and requires no global database snapshot. Chunks can be read at different times, provided each read reflects source state somewhere within its watermark bracket. From the final watermark onward, the reconstructed state continues to track the live source as subsequent change events arrive.

- (2) We prove that the original watermarked algorithm satisfies the contract, and show that other CDC designs fulfill it as well. This covers Debezium’s signal-table modes (`insert_insert` and `insert_delete`), the Debezium 3.6 MySQL and MariaDB read-only variants, the Flink CDC 3.6 MySQL parallel-chunk protocol, log-coordinate-bound reads, and native dumps, establishing that each correctly reconstructs source state.

Our analysis is based on version-pinned documentation and selected source code identified in §7. The executed-GTID proof establishes that sampled GTID sets reliably bound each chunk’s read window when accepted samples represent exact, transaction-complete prefixes of the same history. The parallel proof demonstrates that chunks over disjoint keys require no relative ordering or coordination between their brackets.

- (3) We separate correctness criteria from operational considerations. For correctness, the log must describe one complete history, the copy must partition target keys into non-overlapping chunks, and reconciliation must let later logged changes override stale copied rows. Operational choices (like chunk size, parallelism, bracket width, and whether watermarks require source writes) can affect performance and

throughput, but do not alter the underlying correctness result.

- (4) We verify the framework using both interactive theorem provers and model checking. The full development is machine-checked in Isabelle/HOL, an independent Lean 4 formalization checks the central cut theorems, and five bounded TLA⁺ models exercise the operational protocols and their composition.

This formulation generalizes the virtual cut introduced in earlier work on the classic algorithm [6]. The theory in this paper is self-contained. The mechanizations are independent checks, not prerequisites for reading the proofs.

Reading paths. Practitioners can go directly from this introduction to Section 8. It presents Generalized DBLog in plain operational terms, separating what the database must provide from what the connector must establish. In addition, Table 1 summarizes how production protocols, including Debezium and Flink CDC, fit into this framework.

Readers interested in the formal theory should read §§2–7 in order for the system model, the virtual cut theorems, the reconciliation algorithms, and the connector proofs, followed by §10 for the Isabelle/HOL, Lean 4, and TLA⁺ mechanizations. In addition, Section 9 revisits the original 2020 DBLog paper [1] to make its datastore assumptions explicit, §11 discusses related work, and §12 outlines the scope and limitations of the work.

2 The Setting: Stores, Logs, and Coordinates

We model one *capture source* as a keyed store together with a commit-ordered change log. All definitions are per capture scope, the set of tables or key ranges that one capture run is responsible for.

In the terminology used below, each copied chunk is a *unit*, its two log positions form a *bracket*, and every copied row or observed absence becomes a *read result*. The *frontier* is the position at which we judge the combined output.

The contract answers five questions: (1) which complete, commit-ordered row history is authoritative, (2) which stable row identities the copy must account for, (3) which low and high coordinates bracket each unit’s read, (4) whether replaying the actual emission matches letting the log override the copy, and (5) through which observed coordinate the result is claimed.

The contract alone fixes the canonical replay at the frontier. Emitted streams, transaction closure, and shared witnesses are added in later sections, while physical prefixes, restart, transport, and sink application stay outside the result.

2.1 Keyed stores and commit-ordered logs

Fix a key space K and a value domain V . A *state* is a partial map $\sigma : K \rightarrow V$, where $\sigma(k) = \perp$ means key k is absent. For a multi-table scope, K is the disjoint union of per-table key spaces, each key tagged with its table, and nothing in the theory depends on the tagging.

The elements of K are *stable logical row identities*, not simply values used to order a scan. A primary-key change is normalized as absence for the old identity followed by a value for the new one. A table without a primary key needs another stable identity whose equality is shared by the read and log paths. Ambiguous duplicate rows do not fit the keyed model without an additional normalization that supplies such identities.

The committed history is a finite or growing sequence of *event occurrences* $e_1, e_2, \dots$ in *commit order*. Position supplies occurrence identity. Two occurrences remain distinct even when their keys and images are equal, and we suppress that position tag in the notation. An event occurrence carries a key $\text{key}(e) \in K$ and determines the post-state of that key, $\text{img}(e) \in V \cup \{\perp\}$, with $\perp$ for a delete. This is the row-event shape a log-based CDC deployment configured for full row images consumes, consisting of a binlog or WAL entry carrying the complete after-image of one row, or a tombstone. Events outside the scope are ignored throughout. Logs that carry deltas rather than full images are addressed by S-IMG in §3.2.

Only committed transactions contribute events to this history. An event from an active or aborted transaction is not in the modeled log, cannot be selected by a coordinate or

window, and cannot participate in replay. Source reads that supply copied values likewise observe committed database state. Dirty reads are outside the contract.

State and log are tied together by replay. Writing P for a log prefix, the *row-event replay state* after P is the per-key fold $\sigma_P : \sigma_0$ is the initial state, and $\sigma_P(k)$ is $\text{img}(e)$ for the last event $e \in P$ with $\text{key}(e) = k$, or $\sigma_0(k)$ if no such event exists. Every committed write appears as exactly one event per changed row, so σ_P is the authoritative replay state of those row events at “time” P . Times, here and everywhere below, are log prefixes, never wall clocks. The core deliberately carries no transaction identifier, so a mathematical row-event prefix may end between adjacent events of a multi-row transaction that has already committed and entered the log as one complete block. Such a prefix is only an intermediate state of replaying already-committed row events. It does not represent a partial commit or access to an in-progress transaction. We reserve *transactional snapshot* and *committed database state* for a transaction-closed prefix, one containing either all or none of each transaction’s row events. Any such reading of a theorem below therefore needs transaction closure as an additional instance assumption. The unconditional result is per-key equality with the row-event replay state.

2.2 Mapping coordinates to log prefixes

Real systems do not pass log prefixes around. They pass *coordinates*, such as binlog file/position pairs, LSNs, SCNs, executed-GTID sets, exported snapshot names, and resume tokens. The contract requires only that a coordinate mark a position whose meaning is “these committed row events, and no later ones, lie before here.” A transactional-snapshot claim additionally requires that position to be transaction-closed.

Definition 2.1 (Coordinate space). A *coordinate space* is a set C together with a mapping $\llbracket \cdot \rrbracket : C \rightarrow \text{Prefixes}(\text{Log})$ such that all prefixes in its image are linearly nested, meaning that for any c, c' , either $\llbracket c \rrbracket \subseteq \llbracket c' \rrbracket$ or $\llbracket c' \rrbracket \subseteq \llbracket c \rrbracket$. Write $c \sqsubseteq c'$ for $\llbracket c \rrbracket \subseteq \llbracket c' \rrbracket$. Thus $\sqsubseteq$ is a total preorder on C , and quotienting by equality of mapped prefixes gives a linear order.

Operationally, a usable coordinate lets the capture faithfully answer whether a committed event occurred between two recorded positions.

The only operation the contract ever needs is the *membership oracle*, which, given an event occurrence e and coordinates $c \sqsubseteq c'$, decides whether $e \in \llbracket c' \rrbracket \setminus \llbracket c \rrbracket$, that is, whether e lies inside the window $(c, c']$. No scalar ordering or interval arithmetic over raw coordinate values is required. In practice, scalar LSNs and file/position pairs map to prefixes through their usual comparisons, but executed-GTID sets map to prefixes too, and their membership oracle is set membership on the difference of two GTID sets, with no interval arithmetic

and no scalar anywhere. This modeling choice is what makes read-only capture an ordinary member of the family (§7).

Admissibility is nestedness plus *operational fidelity*: the instance’s window-membership decisions must agree with the declared $\llbracket \cdot \rrbracket$. Raw coordinate comparisons used to establish plan bounds must imply prefix inclusion. Distinct raw coordinates may still map to the same prefix.

A mapping that ignores the system’s actual coordinate behavior proves nothing. Mapping every coordinate to the empty prefix is nested but useless, because a plan that consults coordinates then disagrees with its own declared reading. And a system whose operational coordinates admit *no* mapping to commit-order prefixes that matches their behavior (client-supplied timestamps, sequence numbers allocated at statement time rather than commit time, replicas applying transactions out of commit order) is outside the contract altogether. That is level T0 (§5), the family boundary. Membership is not a matter of degree. Either the system’s coordinates can be read as log prefixes or they cannot. Nestedness itself costs nothing, because mapped prefixes belong to one linear log, and prefixes of a single sequence are always nested. Fidelity is therefore the only substantive admissibility requirement, verified instance by instance (§7).

2.3 Windows and the cutoff discipline

Definition 2.2 (Window). For $lo \sqsubseteq hi$, the *window* $(lo, hi]$ is the ordered event-occurrence slice written $\llbracket hi \rrbracket \setminus \llbracket lo \rrbracket$. Occurrences inherit commit order from the log. Equal key-and-image payloads at different positions remain distinct elements.

Windows are half-open by convention, meaning events at the low edge are excluded and events at the high edge included. Every instance fixes its edges to this convention once. Including or excluding one boundary event twice would duplicate it. Excluding it from both adjacent windows would lose it. The same off-by-one failure can occur at lo or hi , so the convention is part of the formal object. An instance whose raw endpoint records use a different syntax (say, an offset-inclusive $[LOW, HIGH]$ span in a vendor document) must declare its endpoint-to-prefix normalization as an instantiation obligation before its plan may use the window. The semantic window is always $\llbracket hi \rrbracket \setminus \llbracket lo \rrbracket$.

2.4 What stays outside

The contract is source-side. The replay semantics defined in §3 is a mathematical fold over the emitted sequence. It defines what the emitted stream *means*, so that “cut” is a well-defined claim about capture. It is not a delivery-semantics claim. Transport, exactly-once application, sink transactionality, schema and DDL evolution, deployment topology, and security stay outside this paper’s scope and are cited, not treated

(§12). The delivery side of the boundary is the subject of separate work [16]. This boundary has one declared exception, namely the *scope witness*. Determining which relations and keys exist is itself a read that needs a coordinate. Every deployment must therefore satisfy the common *A-scope-witness* condition by naming how the relations and key ranges were enumerated and which coordinate binds that enumeration. The production instances share this common condition. Their individual specifications add only mechanism-specific scope facts. Schema evolution during capture remains out of scope by definition.

3 Capture Plans and the Contract

A capture run has three parts relevant to the proof. It divides the scope into units, reads each unit within a log bracket, and uses an associated merge discipline to emit reads together with log events. The *capture plan* records the final units, brackets, read results, and frontier. It does not record the emitted stream. Scheduling and emission enter separately through the instance and merge obligations.

3.1 Capture plans

Definition 3.1 (Capture plan). A capture plan for scope K at frontier $f \in C$ is a finite family

$$\text{Plan} = \{ (D_i, lo_i, hi_i, R_i) : i = 1..n \}$$

where $\{D_i\}$ partitions K . Each unit carries a *bracket* (lo_i, hi_i) with $lo_i \sqsubseteq hi_i \sqsubseteq f$. The map $R_i : D_i \rightarrow V \cup \{\perp\}$ is the unit’s *refresh*, assigning one value-or-absence read result per key of the unit.

Operationally, K is the captured identity namespace and each D_i is the final ownership predicate for one unit, often a table and half-open key range. These sets are not assumed finite. For a range or table capture they include identities that are currently absent and identities whose rows may be inserted while the capture runs. The finite family of final domains must partition the scope. A finite scan still induces the total map R_i , where returned rows give values and a nonreturned identity indicates implicit absence only when the scan and range predicate establish complete membership. A concurrent insert already belongs to one final domain. Its early inferred absence is valid before the insert, and the observed insert event later overrides it through the log-wins rule.

Brackets may be shared between units (one bracket amortized over many reads), degenerate ($lo_i = hi_i$, a *point bracket*), or global (lo_i at capture start). The frontier f is the coordinate up to which this run makes any claim. Figure 2 draws the anatomy of one unit.

The definition is silent about *how* R_i was obtained, whether by a chunked SELECT, a COPY, a dump file, a restored physical

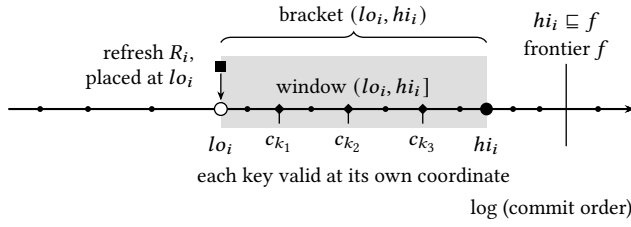


Figure 2: One unit of a capture plan. The bracket (lo_i, hi_i) spans a segment of the commit-ordered log. The window $(lo_i, hi_i]$ is half-open, with events at the low edge excluded and events at the high edge included. The canonical rule places the refresh R_i at lo_i , wherever its rows were actually read. Condition O2 (bracket-local validity) requires less than a single read instant. Each key of the unit reflects the database state at some coordinate of its own inside the bracket $(c_{k_1}, c_{k_2}, c_{k_3})$. Every bracket sits at or below the frontier f .

backup, or a read against a replica. It is also silent about scheduling (how large units are, and whether they ran serially or in parallel) and about how brackets were realized. It carries their endpoints, but imposes no constraint on their numerical width. Scheduling and width do not change the theorem once the contract, observation, and emission-equivalence conditions continue to hold, though either can make those requirements operationally infeasible. Provenance likewise has no field in the tuple, but it can determine which O4 and O5 obligations apply and therefore whether contract membership is established. The plan encodes no physical simultaneity. Corollary 1 instead tests the extensional property that one shared state witness lies in every bracket and agrees with every refresh. An engine snapshot is one way, not the only way, to realize that fact (§4).

Real runs are dynamic. They re-split large chunks mid-flight and discover tables as they go. The definition absorbs this by reading Plan as the *final* partition the run produced. A run that re-splits after it has already emitted reads must satisfy the corresponding equivalence obligation (§3.4) with respect to the final plan. The actually-emitted stream, including any superseded pre-split refreshes, must replay to the final plan’s sink state.

3.2 What the source promises

Three promises bind the source and its log. They carry the 2020 paper’s requirements forward, kept in spirit and sharpened where §9 found them under-specified.

- **(S-LOG: commit-ordered keyed log.)** The log is a linear commit-order event sequence over K . Every committed change to a scoped key appears as exactly one event, and no event from an active or aborted

transaction appears. A committed transaction’s row events become available as one complete block. Key identity is preserved.

- **(S-IMG: image sufficiency.)** $img(e)$ fully determines the post-state of $key(e)$. Where a real log carries deltas, or primary-key updates without the old key, the instance must either strengthen the log (before-images, delete-plus-insert rendering, replica-identity-class settings) or adopt a re-read repair as a *log normalization*. The repaired instance is modeled over effective full-image events, each re-read supplying the image its delta event lacked, and its equivalence obligation then includes that normalization. Absent strengthening or normalization, delta logs are outside this core.
- **(S-OBS: observability.)** The capture process observes every scoped event of $\llbracket f \rrbracket$ from its consumption start s_0 onward, requiring both log coverage and the retention to read what its merge discipline consumes. Where s_0 must sit, and in what access mode window ranges must be readable, is owned by the discipline in use, and each discipline’s definition states its own bound (§6). A single pass from $s_0 \sqsubseteq lo_i$ for every unit suffices for window-discard, while range-merge streams from $s_0 \sqsubseteq hi_i$ and reads windows by coordinate-addressable access.

3.3 Capture plan requirements

- **(O1: coverage.)** $\{D_i\}$ partitions K , so every scoped key is owned by exactly one unit. Plans whose units overlap are outside the core contract. A composition with overlapping branches must satisfy the coherence obligation that the branches replay identically, and this paper treats only partitions.
- **(O2: bracket-local validity.)** For every unit i and every key $k \in D_i$ there exists a coordinate c with $lo_i \sqsubseteq c \sqsubseteq hi_i$ such that $R_i(k) = \sigma_{\llbracket c \rrbracket}(k)$.
- **(O3: domain completeness.)** R_i is total on D_i , so that every key of the unit yields exactly one value-or-absence result.
- **(O4: declared external assertions.)** Where a bracket coordinate is asserted by tooling rather than constructed or observed by the capture process (dump metadata, a backup’s recorded recovery point), the coincidence “ R_i was read at the asserted coordinate” enters as an *explicit assumption* of the instance, never as a proved fact.
- **(O5: representation agreement.)** The refresh path and the log path agree on key identity and value encoding. Formally, both land in the same K and V . Operationally, a checkable clause that applies exactly

when the refresh bypasses the connector’s row path, as dumps and backups do.

O2 is the central weakening of this paper. It is *per-key* and *existential*. Different keys of one unit may reflect the source at different in-bracket coordinates, and the contract does not require one shared read coordinate. Many concrete reads satisfy O2 more strongly, as when a statement snapshot or coordinate-bound read supplies the same read coordinate for every key in its unit. The classic algorithm’s single-coordinate chunk reads satisfy O2 in this way (§7). The per-key form also admits a complete refresh whose keys reflect different read coordinates. Section 4.3 constructs such an instance and proves that no single coordinate explains all of its read results, while its cut still holds.

The database operation that supplies R_i is a committed read. When an instance retains transaction labels, its operational read coordinate lies at a transaction boundary. The executed-set instance of §7.4 makes this explicit. The transaction-free core records only the per-key equality required by the proof. It does not turn an interior row-event replay prefix into an allowable dirty-read state.

Together, O2 and O3 pin window-invariant keys exactly. If no in-window event touches k , then all candidate values $\sigma_{[c]}(k)$ across the bracket coincide, so the read result $R_i(k)$ is that invariant value.

A scan that silently misses a key cannot satisfy both obligations, because absence-by-omission is sound only when the key is genuinely absent. This pair, not read freshness and not a specific isolation level, is the property a scan must preserve. Section 9 returns to the point, because the 2020 paper’s own requirement was stated in freshness vocabulary and the gap between the two formulations is where one class of engines lacks statement-stable scan membership.

O4 makes external trust explicit. Splice-style instances use coordinates that arrive from outside the capture process, such as a coordinate written into a header by a dump tool or a recovery point recorded by a backup system. The contract does not treat these as observations. Each such reliance becomes an explicit assumption that the instance records visibly (§7), so that every level or certification claim in this paper is readable as “proved, given this list.” Where the list is empty, no additional external coordinate assertion is assumed. Otherwise the reader can check each entry against their own deployment.

Definition 3.2 (Source-side contract). A source, a coordinate space, and a capture plan *satisfy the contract* iff S-LOG and S-IMG hold and the plan satisfies O1-O5.

S-OBS stands outside the definition for a reason. Within the model, S-LOG and S-IMG specify the commit-ordered full-image event sequence of §2. An instantiation over a real system carries them as that modeling claim. The contract and

its cut theorem constrain the *canonical* replay, a fold over the committed history directly (§3.4), and require no observation clause. What a capture process actually observed becomes decisive at the point where an emitted stream is claimed equivalent to the canonical one, and there each discipline’s equivalence theorem relies on S-OBS in the discipline’s own form (§6). A deployment must satisfy all three source requirements. The split records which theorems depend on which clauses.

3.4 Canonical replay and emission equivalence

For canonical *replay*, we place each unit’s refresh at lo_i and let subsequent log events override it. At the frontier, this gives the per-key fold: for $k \in D_i$,

$$sink_f(k) = \begin{cases} img(e^*) & \text{if } e^*, \text{ the last event for } k \text{ in} \\ & (lo_i, f], \text{ exists,} \\ R_i(k) & \text{otherwise.} \end{cases}$$

Placing refreshes at lo_i and letting later events win is the *canonical merge rule*. The refresh overrides events at or before lo_i , and events in $(lo_i, f]$ override the refresh. The low edge is a conservative placement. Condition O2 permits each read result to reflect the source state at some coordinate within the bracket, while the fold gives every later in-window and post-window event final precedence. It is a definition of meaning, not an implementation prescription, and shipped systems do not need to emit this stream. How far a real discipline departs from it varies. The classic algorithm buffers a chunk and *discards* buffered rows as in-window events arrive. Flink-shaped systems *merge* the window’s events onto the chunk and emit the unit once. Splices emit a dump behind a point bracket with nothing to reconcile. Their modeled copy-before-suffix stream is replay-equivalent to the canonical rule when that order, or an equivalent merge, is preserved. Each such discipline M is admitted through an *equivalence obligation*. The instance must show that M ’s emitted stream, built from what the capture observes under M ’s own S-OBS form, replays to the same $sink_f$ as the canonical rule. Usually that is a short proof, and §6 is a catalog of the established ones, alongside entries whose obligations are stated but left open, labeled as that. This proof approach makes “the log wins” a theorem about the canonical rule rather than a separate argument per implementation. Several industrial correctness conditions, such as withholding rules and buffer-close semantics, surface at this layer (§6).

The next section establishes the contract guarantee. For every plan satisfying Definition 3.2, the canonical replay $sink_f$ is exactly the source’s row-event replay state $\sigma_{[f]}$, on every key, at the frontier, regardless of provenance, parallelism, or width.

4 The Cut Theorem and Its Corollaries

The contract yields one theorem, a derived latest-high lemma, a closed-unit lemma, and three corollaries. The theorem states that a conforming plan’s canonical replay equals the source’s row-event replay state at its frontier. The corollaries carry the readings a practitioner needs beyond the frontier value itself. They show how an exhibited shared state witness can move the canonical trajectory earlier (Corollary 1), what a capture may claim past the point where it stopped observing (Corollary 2), and why plans assembled from different capture methods compose into one guarantee (Corollary 3). The theorem’s proof and the first two corollaries’ are local to one key and its one covering unit, and the third corollary composes whole plans without ever coupling their units. That locality is itself a result, and Section 7 relies on it.

4.1 Window invariance and the theorem

The entire development rests on one lemma. It says that if nothing happened to a key inside a window, then the key’s value is the same at every coordinate of that window. Hence a read that matches the source state at any coordinate in such a window matches it at every coordinate in that window. Bracket width therefore never enters the canonical cut theorem. Widening a bracket enlarges the set of events the merge must reconcile, and can strain its observation requirements, but for the keys the refresh actually decides, those with no event in the window, every coordinate of the bracket is as good as every other.

Lemma 4.1 (Window invariance). *Let $lo \sqsubseteq c \sqsubseteq hi$ and let k be a key with no event in the window $(lo, hi]$. Then $\sigma_{[c]}(k) = \sigma_{[lo]}(k)$. Consequently $\sigma_{[c]}(k) = \sigma_{[c']}(k)$ for any two coordinates c, c' of the bracket.*

PROOF. The prefix $[c]$ extends $[lo]$ by exactly the window $(lo, c]$, and $(lo, c] \subseteq (lo, hi]$ because $c \sqsubseteq hi$. The extension therefore contains no event for k , and the per-key fold of §2.1 leaves $\sigma(k)$ unchanged across an extension without events for k . The consequence follows by applying this to c and to c' and comparing both against lo . $\square$

Theorem 1 (Cut theorem). *Let a source, a coordinate space, and a capture plan at frontier f satisfy the contract (Definition 3.2). Then the canonical replay equals the source’s row-event replay state at the frontier on every key of the scope:*

$$\text{sink}_f(k) = \sigma_{[f]}(k) \quad \text{for all } k \in K.$$

PROOF. Fix $k \in K$. By O1 exactly one unit i owns k , and both sides of the claim are decided by the events for k alone. The left side consults the window $(lo_i, f]$ and the refresh $R_i(k)$, while the right side is the image of the last event for k in $[f]$, or $\sigma_0(k)$ if there is none. Since $lo_i \sqsubseteq hi_i \sqsubseteq f$, the prefix $[f]$ consists of $[lo_i]$ followed, in commit order, by the

window $(lo_i, f]$. The events for k in $[f]$ split the same way. We distinguish whether the window part is empty.

Case A: some event for k lies in $(lo_i, f]$. Let e^* be the last such event. The replay returns $\text{sink}_f(k) = \text{img}(e^*)$ by definition. By the split, e^* is also the last event for k in all of $[f]$, so $\sigma_{[f]}(k) = \text{img}(e^*)$ as well. The fold discards the refresh in favor of e^* regardless of what the read returned for k . Condition O2 for $R_i(k)$ is not needed here, only its existence (O3), and the eviction behavior of every real merge discipline (§6) implements this property of the fold.

Case B: no event for k lies in $(lo_i, f]$. The replay returns $\text{sink}_f(k) = R_i(k)$, and by O3 that read exists and is unique. O2 supplies a coordinate c with $lo_i \sqsubseteq c \sqsubseteq hi_i$ and $R_i(k) = \sigma_{[c]}(k)$. The window $(lo_i, f]$ contains no event for k , and $lo_i \sqsubseteq c \sqsubseteq f$ since $c \sqsubseteq hi_i \sqsubseteq f$. Two applications of Lemma 4.1, with f as the window’s high edge, therefore give

$$R_i(k) = \sigma_{[c]}(k) = \sigma_{[lo_i]}(k) = \sigma_{[f]}(k),$$

which is the claim. $\square$

Two remarks on the proof’s shape. Both are used later.

The frontier is parametric. Beyond the bracket bounds $lo_i \sqsubseteq hi_i$, the argument relied on exactly one property of f , namely that $hi_i \sqsubseteq f$ for the covering unit. The identical argument therefore proves replay to *any* coordinate g bounding every unit’s high edge equal to $\sigma_{[g]}$. This parametric form is what Lemma 4.2 and Corollary 2 instantiate.

Nothing couples distinct units. The proof fixes one key, finds its one unit, and never mentions another. No condition couples two units’ brackets, no step compares their read coordinates, and no bound on how many units run concurrently appears anywhere. Nothing couples distinct units beyond O1 disjointness and the shared frontier bound. Whether the units ran serially or in parallel, and whether their brackets were disjoint, nested, or identical on the log axis, is invisible to the argument. Corollary 3 applies this observation, and the parallel-chunks instance of Section 7 derives its guarantee from it without any coordination argument of its own.

Frontier equality is per-key. The equality of Theorem 1 holds key by key at the frontier. Each replayed value equals that key’s value in the row-event replay state at the frontier. The core framework assumes nothing about transaction boundaries. Consequently, calling that state a transactional snapshot additionally requires the frontier to be transaction-closed. Corollary 1’s trajectory requires the same closure at every coordinate given that reading. The contract alone supplies neither closure nor physical sink-prefix behavior. Section 8.4 gives a concrete practitioner example of the distinction between committed-only input and a transaction-closed result. Where coordinates are transaction boundaries, as with the executed-transaction sets of §7.4, that stronger structure enters through instance-specific assumptions.

Monotonicity is discipline-scoped. Theorem 1 is a statement about the final replayed state. Whether the sink also moves *monotonically* while the emission is consumed, never showing an older value for a key after a newer one, depends on the merge discipline and is proved per discipline in §6. Under the canonical rule itself, monotonicity holds for every window-invariant key. A key touched inside its window can step backward transiently. Its refresh is placed at lo_i even when the read that produced it happened late in the bracket, so replay may apply that late value first and the key’s earlier in-window events after it, until the last event restores the final value. The transient is confined to the window that produced it, and the final value is unaffected. The two production disciplines both eliminate it. Window-discard emits a key’s events in commit order with the surviving refresh placed at the window close, and range-merge emits the merged state for each key at the close followed only by later events. Both per-key monotonicity theorems are stated and proved in §6.

4.2 A worked replay

Executing the theorem’s fold once by hand shows both the log winning and what the refresh is actually for.

Example 4.1 (Window replay). Take three keys a, b, c over an initially empty store (σ_0 assigns $\perp$ everywhere), and a committed history of three events

$$\begin{aligned} L = e_1 e_2 e_3, \quad & e_1 : \text{key}(e_1) = a, \text{ img}(e_1) = 1, \\ & e_2 : \text{key}(e_2) = a, \text{ img}(e_2) = 2, \\ & e_3 : \text{key}(e_3) = c, \text{ img}(e_3) = 9. \end{aligned}$$

Coordinates c_0, c_1, c_2, c_3 map to the prefixes of lengths 0 through 3. The plan has one unit, with domain $\{a, b, c\}$, bracket $(lo, hi) = (c_1, c_2)$, frontier $f = c_3$, and refresh

$$R(a) = 1, \quad R(b) = \perp, \quad R(c) = \perp,$$

a read that matches the source state at the low edge c_1 . After e_1 , key a held 1 and keys b, c were absent, so O2 is satisfied with witness c_1 for all three keys, and O3 holds since R gives a read result for each. The replay to f works each key through the window $(c_1, c_3] = e_2 e_3$:

- *Key a (a stale read, evicted).* The window contains e_2 , so Case A applies, giving $\text{sink}_f(a) = \text{img}(e_2) = 2$. The refresh value 1 was true when the cursor passed a and is stale at the frontier. The fold never consults it. This is “the log wins.”
- *Key b (true absence, kept).* No event for b exists anywhere, so Case B applies and $\text{sink}_f(b) = R(b) = \perp$, which is correct because O2 forces the recorded absence to be valid somewhere in the bracket and Lemma 4.1 extends it to f .
- *Key c (superseded after the bracket).* The window $(c_1, c_3]$ contains e_3 , which committed after the high

edge c_2 but at or before the frontier. Case A applies, yielding $\text{sink}_f(c) = 9$. The refresh recorded c as absent, validly inside the bracket, yet the later event supersedes it, as a live stream should.

All three agree with $\sigma_{\llbracket c_3 \rrbracket} = \{a \mapsto 2, c \mapsto 9\}$, as Theorem 1 requires.

4.3 Per-key validity is strictly weaker than a single read point

Section 3.3 called O2 the central weakening. The following instance shows that the weakening is strict. Its refresh combines two committed observations made at different coordinates. No single coordinate accounts for both reads, yet the contract admits the instance and its cut still holds.

Proposition 1 (No shared read point). *There is a contract instance with one unit over two keys such that*

- (1) *no single coordinate c in the unit’s bracket satisfies $R(k) = \sigma_{\llbracket c \rrbracket}(k)$ for both keys k , yet*
- (2) *the instance satisfies O1–O5, and*
- (3) *its canonical replay equals the row-event replay state at the frontier.*

PROOF. Over an initially empty store, take two keys a, b and the two-event history $L = e_1 e_2$ with $\text{key}(e_1) = a$, $\text{img}(e_1) = 1$, $\text{key}(e_2) = b$, $\text{img}(e_2) = 2$, and coordinates c_0, c_1, c_2 mapping to the prefixes of lengths 0, 1, 2. The plan has one unit: domain $\{a, b\}$, bracket (c_0, c_2) , frontier $f = c_2$, and refresh

$$R(a) = \perp, \quad R(b) = 2.$$

The read of a happened before e_1 committed and the read of b after e_2 committed. Both observations are of committed state, but they reflect different source coordinates.

Contract. O1 holds with the single unit. O3 holds since R is total. O5 holds by construction. For O2, key a matches the state at c_0 , where $\sigma_{\llbracket c_0 \rrbracket}(a) = \perp = R(a)$, and key b matches the state at c_2 , where $\sigma_{\llbracket c_2 \rrbracket}(b) = 2 = R(b)$. Both witnesses lie in the bracket (c_0, c_2) .

No single shared read coordinate. The bracket contains exactly the coordinates c_0, c_1, c_2 . At c_0 and c_1 the source has $\sigma(b) = \perp \neq 2 = R(b)$. At c_2 it has $\sigma(a) = 1 \neq \perp = R(a)$. No in-bracket coordinate is consistent with both keys, so an obligation demanding one read point per unit rejects this plan.

Cut. The window $(c_0, c_2]$ contains e_1 and e_2 , so both keys fall under Case A of Theorem 1, with $\text{sink}_f(a) = \text{img}(e_1) = 1$ and $\text{sink}_f(b) = \text{img}(e_2) = 2$, which is exactly $\sigma_{\llbracket c_2 \rrbracket}$. The refresh results do not decide the final fold. $\square$

The proposition also shows why the classic algorithm’s single-coordinate chunk reads satisfy this contract directly. A

single read point implies O2 by instantiating every key's witness uniformly, while O2 does not imply a single read point. The weakening is strict, and it is the exact degree of freedom needed when an implementation establishes per-key bracket condition O2 without establishing a single shared read coordinate. The distinction is between proof obligations. Ordinary chunk selects may well run under statement snapshots. The read-only and parallel instances of Section 7 are placed without assuming one, since the isolation of their chunk selects is undocumented (§12).

4.4 Corollary 1: a shared witness can move the canonical trajectory earlier

The frontier-parametric proof constrains more than one state. To make that fact and the additional role of a shared witness precise, we first extend the canonical replay to intermediate coordinates.

Definition 4.1 (Canonical replay at a coordinate). For any coordinate g and key $k \in D_i$, the *canonical replay at g* is

$$\text{sink}_g(k) = \begin{cases} \text{img}(e^*) & \text{if } e^*, \text{ the last event for } k \text{ in} \\ & (lo_i, g], \text{ exists,} \\ R_i(k) & \text{otherwise,} \end{cases}$$

where $(lo_i, g]$ is read as $\llbracket g \rrbracket \setminus \llbracket lo_i \rrbracket$ for an arbitrary pair of coordinates. When $g \sqsubseteq lo_i$ it is empty and the refresh stands. We call the family $g \mapsto \text{sink}_g$ the plan's *canonical trajectory*.

At $g = f$ this is the canonical replay of §3.4. For general g it is a retrospective family derived from the final plan. Every unit contributes its refresh results, and events up to g supersede them. It is not, by definition, the sequence of states produced while an emitted stream is consumed. Calling those physical sink prefixes a trajectory would require prefix-equivalence at every g , or an atomic or ordered handoff condition, beyond the final equivalence obligation of §3.4.

Lemma 4.2 (Latest-high trajectory). *Let a nonempty finite capture plan at frontier f satisfy the contract. Among its recorded high edges choose one h whose corresponding prefix is maximal. Then $h \sqsubseteq f$, every unit satisfies $hi_i \sqsubseteq h$, and for every g with $h \sqsubseteq g \sqsubseteq f$ and every $k \in K$,*

$$\text{sink}_g(k) = \sigma_{\llbracket g \rrbracket}(k).$$

Thus the canonical trajectory of every nonempty T2 plan matches the source from its latest high edge onward.

PROOF. The unit family is finite and coordinates map to linearly nested prefixes, so the nonempty set of prefixes at its high edges has a maximum. Choose a recorded high edge h that maps to that prefix. Every $hi_i \sqsubseteq h$, and $h \sqsubseteq f$ because every unit high is bounded by the frontier. For any g with $h \sqsubseteq g \sqsubseteq f$, transitivity gives $hi_i \sqsubseteq g$ for every unit. The

frontier-parametric form of Theorem 1 therefore applies at g . $\square$

If the unit family is empty, O1 forces the capture scope to be empty. The scoped equality is then vacuous, but there is no recorded high edge to select. The rest of the paper concerns nonempty captures.

The proof needs the high-edge bound only for the unit that owns the current key. This yields the mid-capture form illustrated in Figure 6.

Lemma 4.3 (Closed-unit cut). *Let a source, a coordinate space, and a capture plan at frontier f satisfy the contract, let g be any coordinate, and let $k \in D_i$ be a key whose unit has closed by g , that is, $hi_i \sqsubseteq g$. Then $\text{sink}_g(k) = \sigma_{\llbracket g \rrbracket}(k)$. Consequently, at every coordinate g , the canonical replay agrees with the row-event replay state on every key owned by a unit whose high edge lies at or before g , whether or not the other units have closed.*

PROOF. The proof of Theorem 1 fixes k and its covering unit i and uses the frontier only through $hi_i \sqsubseteq f$. With g in place of f and Definition 4.1 supplying sink_g , Case A and Case B apply verbatim. The last event for k in $(lo_i, g]$ decides both sides, or, if there is none, O2 and Lemma 4.1 carry the valid state from the bracket to g because $c \sqsubseteq hi_i \sqsubseteq g$. No fact about any other unit is used. $\square$

Figure 6 illustrates the lemma. Once a unit closes, its keys are exact at the close and remain exact as later events are replayed, while keys in open units carry no claim. Under their respective observation and closure conditions, the same per-key argument applies to the corresponding prefixes of the window-discard and range-merge emissions (Lemmas 6.1 and 6.3).

Corollary 1 (Shared witness). *Let a source, a coordinate space, and a capture plan at frontier f satisfy the contract, and suppose some coordinate c^* is a shared state witness for the whole plan:*

- $lo_i \sqsubseteq c^* \sqsubseteq hi_i$ for every unit i , and
- $R_i(k) = \sigma_{\llbracket c^* \rrbracket}(k)$ for every unit i and every $k \in D_i$, meaning that each refresh matches the source state at c^* , not merely somewhere in its own bracket.

Then for every g with $c^ \sqsubseteq g \sqsubseteq f$ and every $k \in K$,*

$$\text{sink}_g(k) = \sigma_{\llbracket g \rrbracket}(k) :$$

the canonical replay trajectory coincides, key by key, with the source's row-event replay trajectory from c^ up to the observation frontier.*

PROOF. Fix g with $c^* \sqsubseteq g \sqsubseteq f$, fix k , and let i be the covering unit. Note $lo_i \sqsubseteq c^* \sqsubseteq g$. If some event for k lies in $(lo_i, g]$, the last such event decides both sides exactly as in Case A of Theorem 1, with g in place of f . Otherwise

the window $[lo_i, g]$ contains no event for k . Since $(c^*, g] \subseteq [lo_i, g]$, the window $(c^*, g]$ contains no event for k either, and Lemma 4.1 gives $\sigma_{[g]}(k) = \sigma_{[c^*]}(k)$. Hence

$$\text{sink}_g(k) = R_i(k) = \sigma_{[c^*]}(k) = \sigma_{[g]}(k),$$

using the shared state witness for the middle equality. $\square$

The additional information is the *exhibited common state witness*. Lemma 4.2 already gives every nonempty T2 plan a conservative trajectory from its latest high h . Corollary 1 identifies one coordinate c^* at which every refresh agrees with the same source state. Because $c^* \sqsubseteq hi_i \sqsubseteq h$ for every unit, it can move the guaranteed start earlier than h . The start is not necessarily earlier, since c^* may equal h . We call it the plan's *exhibited shared-state anchor*, and T3 records this additional evidence (§5).

The formal plan does not place it before the physical reads or claim that physical sink prefixes follow this history. Transactional-history language additionally requires transaction closure throughout the claimed range. Beyond f the canonical trajectory extends as far as observation is extended (Corollary 2).

Operationally, a shared state witness arises in three ways. The first is a shared point bracket, where $lo_i = hi_i = c^*$ for every unit and the windows are empty, so there is nothing to merge and the splice determines everything. The second is a shared engine snapshot read inside wider brackets, as when every unit's select runs against one exported or MVCC read view. The third is a quiesced window, where writes are simply held off between the reads. The splice instances of §7 are the first case as it occurs in production.

4.5 Corollary 2: continuation under extended observation

Corollary 2 (Continuation). *Let a source, a coordinate space, and a capture plan at frontier f satisfy the contract, and let $f \sqsubseteq f'$. Then $\text{sink}_{f'}(k) = \sigma_{[f']}(k)$ for every $k \in K$.*

PROOF. Every unit satisfies $hi_i \sqsubseteq f \sqsubseteq f'$, so f' bounds every bracket's high edge, and the frontier-parametric form of Theorem 1's proof applies verbatim with f' in place of f . $\square$

The extension costs observation. The corollary's fold is a statement about the committed history. A capture *realizes* it as far as it keeps observing, which is S-OBS extended to f' . The fold beyond the frontier consumes events a stopped capture no longer sees, and no algebra recovers them. Steady-state operation, where a bootstrap completes and the pipeline then follows the log indefinitely, is Corollary 2 applied at ever-larger f' . The guarantee survives as long as consumption and retention do.

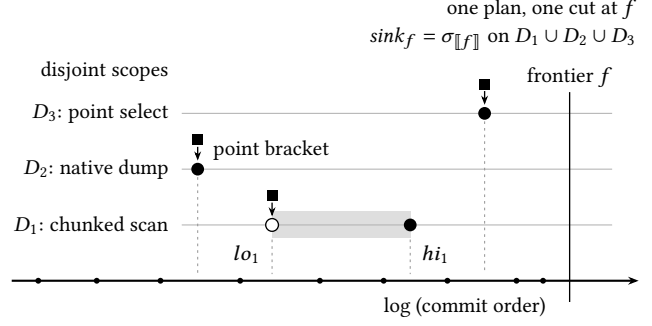


Figure 3: A heterogeneous plan at one frontier. Three units over disjoint scopes mix provenances, including a chunked scan with an ordinary bracket, alongside a native dump and a point select each behind a point bracket of its own. No coordinate lies inside all three brackets, so no shared state witness is available. The composite nevertheless satisfies the contract. One cut at f covers the union of the scopes, and its canonical trajectory is guaranteed from the latest of the three high edges (Corollary 3, Theorem 1, and Lemma 4.2).

4.6 Corollary 3: mixing and composition guarantees

Nothing in Definition 3.1 records how a refresh was obtained, so a single plan may already mix a chunked scan here, a dump there, and a point select elsewhere (Figure 3). Theorem 1 quantifies over all of them at once. Heterogeneous bootstrap is therefore not a special case requiring its own theory. It is the theorem's normal case, and the only composition fact left to check is that conforming plans over disjoint scopes can be pasted into one conforming plan.

Corollary 3 (Mixing). *Let two capture plans over the same source and coordinate space satisfy the contract at the same frontier f , on disjoint scopes K_1 and K_2 . Then their union, with units combined and scope $K_1 \cup K_2$, satisfies the contract at f , and consequently replays to $\sigma_{[f]}$ on all of $K_1 \cup K_2$.*

PROOF. Each obligation decomposes over the union. For coverage, the combined domains cover $K_1 \cup K_2$ because each family covers its own scope. For disjointness, two units from the same plan have disjoint domains by that plan's O1. Two units from different plans have disjoint domains because each domain is contained in its plan's scope and the scopes are disjoint. Brackets and their frontier bounds are per-unit facts and carry over unchanged, as do O2's per-key witnesses, O3's totality, and O5's typing. O4's declared assumptions combine by-union, and S-LOG and S-IMG are facts of the one shared source and log, common to both plans. The union therefore satisfies Definition 3.2, and Theorem 1 applied to it gives the cut on the combined scope. $\square$

The same argument gives one more decomposition. If a single coordinate c^* is a shared state witness for *every* member of both plans, then it is a shared state witness for the union, since the two hypotheses of Corollary 1 are per-unit statements. The composite then carries the full canonical trajectory property from c^* . Sharing one engine snapshot across all members, its coordinate recorded, lifts the composite to T3.

Without a witness shared by every member, composition alone supplies the T2 cut. When the combined unit family is nonempty, it also supplies the conservative trajectory from the union’s latest high. It does not automatically supply one shared-state anchor or an earlier start. A shared witness may still exist for a particular plan, but it needs separate evidence. The sharpness obstruction has the exact shape of per-table initial loads taken at distinct coordinates. Each table’s copy matches that table’s row-event replay state at its own coordinate, but the union may fail to match the combined scope at any one coordinate. We call this the *tablesnc* shape. Section 5 categorizes the real systems that have it.

Proposition 2 (Composition can lose every earlier onset). *There is a contract instance composed of two units, each of which, as a one-unit plan on its own scope, satisfies the hypotheses of Corollary 1 at its own witness, while the composite has no shared in-bracket coordinate and admits no early onset. For every coordinate c^* strictly below the frontier, the trajectory property of Corollary 1 fails at some g with $c^* \sqsubseteq g \sqsubseteq f$.*

PROOF. Over an initially empty store, take the history $L = e_1 e_2$ with $\text{key}(e_1) = a$, $\text{img}(e_1) = 1$, $\text{key}(e_2) = b$, $\text{img}(e_2) = 2$, coordinates c_0, c_1, c_2 as in Proposition 1, and frontier $f = c_2$. Compose two point-bracket units:

$$\begin{aligned} A : \text{domain } \{a\}, \text{ lo}_A = \text{hi}_A = c_0, R_A(a) = \perp, \\ B : \text{domain } \{b\}, \text{ lo}_B = \text{hi}_B = c_2, R_B(b) = 2. \end{aligned}$$

Unit A reads before anything commits and records a absent. Unit B reads at the end of history and records $b = 2$. Each read is valid at its unit’s point coordinate, so each one-unit plan satisfies the hypotheses of Corollary 1 with witness c_0 respectively c_2 , and the composite satisfies the contract by Corollary 3. Theorem 1 gives its cut at c_2 .

The point brackets $[c_0, c_0]$ and $[c_2, c_2]$ have no coordinate in common, so the composite has no shared in-bracket witness. Its latest high is $h = c_2 = f$, and Lemma 4.2 gives the conservative, degenerate trajectory interval $[f, f]$.

Now evaluate the composite’s trajectory at the intermediate coordinate $g = c_1$. Unit B ’s window $(c_2, c_1]$ is empty, so $\text{sink}_{c_1}(b) = R_B(b) = 2$, although $\sigma_{[c_1]}(b) = \perp$. The canonical replay asserts a value for b absent from the source’s row-event replay at c_1 . The trajectory property therefore fails at $g = c_1$, and since any onset c^* strictly below the frontier satisfies $c^* \sqsubseteq c_1 \sqsubseteq c_2 = f$, no such onset exists. $\square$

The resulting composition rule is short. Every conforming composite reaches T2 by Corollary 3 and Theorem 1. For a nonempty combined unit family, Lemma 4.2 also supplies its conservative trajectory. Proposition 2 shows that composition alone cannot promise a shared witness or any earlier onset. One exhibited witness shared by all members establishes T3 (Corollary 1 via the decomposition above). Bracket *sharing*, by contrast, is unconstrained by the contract. Definition 3.1 already allows distinct units to carry identical brackets, no obligation counts brackets, and one bracket amortized over many units with one shared state witness is exactly the hypothesis of Corollary 1. Amortizing a single bracket over many reads is therefore level-preserving, which is what makes batching brackets a pure cost optimization (§7).

5 Guarantee Levels and Certification

Levels T0 through T3 describe what has been established about a configuration’s replay and witness structure. T4 records a different dimension, whether a T2 or T3 claim carries an externally checkable certificate. It is therefore a certification overlay rather than a stronger replay semantics. Each production shape in §7 receives the applicable semantic level and, when available, the certification overlay. Mechanisms not placed here remain explicitly open.

These levels separate what can be *proved* from how a configuration performs. Most knobs operators turn do not change either dimension. Figure 4 draws the semantic progression, the independent certification path, and the cost knobs that belong to neither.

Operationally, T2 means that the plan’s canonical replay is correct at its frontier and, after the latest recorded high edge, along its canonical trajectory. An implementation’s emitted stream achieves that result only after its equivalence proof is established. T3 additionally exhibits one coordinate at which all copied units agree with the same source state, which may move the trajectory start earlier. T0 and T1 establish no source-side cut.

5.1 Four semantic levels and one certification overlay

- **T0: outside the contract.** The configuration is not an instance. Either no faithful prefix reading of its coordinates exists (the admissibility clause of §2.2, where the system’s own membership decisions agree with no mapping to commit-order prefixes), or S-LOG, S-IMG, or a plan obligation provably fails with no declared repair. Examples include deletes invisible to the consumed stream or an enumeration provably incomplete. Note what T0 is not. A lawful plan with a wide bracket, or even one run-global bracket, is still

an instance. Width never moves a plan to T0, and a global bracket is still a bracket.

- **T1: cut not established.** The configuration’s guarantee argument is incomplete as shipped. Bracket-local validity, domain completeness, or the merge discipline’s equivalence obligation (§6), its S-OBS floor included, is not established. The framework then claims nothing beyond the emission’s content, establishing neither a cut nor per-key monotonicity. Such deployments may nevertheless converge by their own unproved mechanism or through machinery external to this contract, such as version-stamped apply and checksum repair. That machinery is sink-side and stays outside the modeled scope (§12). Dump-then-verify pipelines can live here, as do backfill-skipping modes whose own documentation promises at-least-once delivery only (§7). The level judges the argument, not the software. A buffering service whose equivalence obligation is undischarged sits at T1, and it moves to T2 as soon as the obligation is satisfied.
- **T2: per-key replay equivalence at the frontier.** At Theorem 1’s level, canonical replay equals the source’s row-event replay state at the frontier, key by key, on the whole scope. Every contract instance reaches T2 as a plan, whatever its provenance, parallelism, or bracket geometry. The cut theorem requires nothing else. A shipped configuration stands here once its plan satisfies the contract and its discipline’s equivalence obligation is established (the second choice of §5.2). With the equivalence undischarged, the same deployment is the T1 case above. Reading this row-event cut as a transactional snapshot additionally requires a transaction-closed frontier. For every nonempty plan, Lemma 4.2 also guarantees the canonical trajectory from the latest recorded high edge through the frontier.
- **T3: exhibited shared-state anchor.** This is Corollary 1’s level, claimed at a named coordinate. Some c^* lying inside every unit’s bracket is a shared state witness, *exhibited by the capture* through a recorded binding between the copied state and that coordinate. From c^* onward, the plan’s canonical trajectory coincides with the source’s row-event replay. Because $c^* \sqsubseteq hi_i$ for every unit, this can begin earlier than T2’s conservative latest-high start. Equality is possible, so the semantic interval is not strictly longer in every instance. The additional guarantee is the shared-state evidence itself. This level does not assert that physical reads were simultaneous or that transient prefixes of an emitted stream follow that trajectory. The latter requires prefix-equivalence or an atomic, ordered handoff. A transactional-history

reading additionally requires transaction closure at the coordinates considered.

- **T4: certification overlay.** T2 or T3 together with an externally checkable certificate whose acceptance supports the cut, provided the checker assumptions hold and the required source history and chunk reads are recorded correctly. Certification neither creates a shared witness nor moves the trajectory start. The certified virtual-cut object of the classic algorithm’s published formalization [6] anchors this overlay and is the formalized example used here. Section 7 anchors the T4 overlay there.

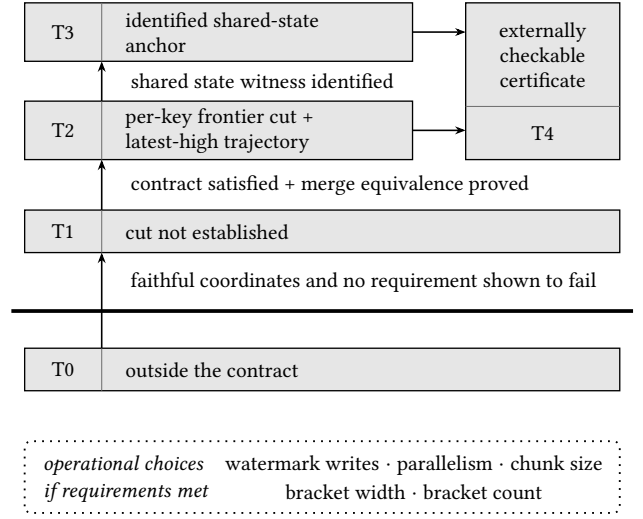


Figure 4: Guarantee structure. Three choices determine the semantic level. A faithful coordinate reading, with no disproved source promise or plan obligation, crosses the T0 exclusion boundary. Fulfilling all contract obligations together with a proven merge equivalence guarantees the cut for the emitted stream. An identified shared state witness adds one shared-state anchor and may move the conservative start earlier. Certification applies independently to either T2 or T3. Holding those conditions fixed, the operational choices change neither semantic level nor certification.

5.2 What changes a level, and what does not

Three choices determine the semantic level:

- (1) a faithful coordinate reading, with neither S-LOG, S-IMG, nor any plan obligation disproved without a declared repair (the T0 exclusion boundary).
- (2) all contract obligations, including bracket-local validity, domain completeness, and image sufficiency, plus a proven merge-discipline equivalence (T1 versus T2).

(3) an exhibited shared state witness (T2 versus T3).

Certification is a separate yes-or-no dimension. An externally checkable certificate adds the T4 overlay to either T2 or T3 without changing the underlying replay or witness claim.

Holding the contract, S-OBS, and emission-equivalence conditions fixed, write-freeness, parallelism, chunk size, bracket width, and bracket count affect cost rather than theorem strength. Provenance and scheduling appear nowhere in the plan signature, and no theorem condition depends on the width of the bracket pair. A wider bracket can, however, enlarge reconciliation work and the log span that S-OBS must keep readable. Cost choices can therefore determine whether a deployment can actually satisfy the required conditions and establish a level, even though the theorem does not rank them directly.

5.3 Reading the levels on a mixed bootstrap

Example 5.1 (Composition across levels). An operator bootstraps four tables. Tables t_1, t_2, t_3 are loaded from per-table dumps, each a point bracket at its own recorded coordinate. Table t_4 is chunk-scanned with ordinary brackets. Each dump alone satisfies the hypotheses of Corollary 1 on its own scope at its own coordinate, so each is T3 on its own table. The composite of all four is a contract instance by Corollary 3 and reaches T2 on the union, with one virtual cut at the shared frontier and a conservative trajectory from the latest recorded high. The member witnesses do not automatically yield T3 on the union. The three dump coordinates are distinct, and Proposition 2 exhibits this shape with no shared witness and no onset before the frontier. This is the *tablesync* shape, named after the per-table initial-copy workers of logical replication [17]. Every table matches its own coordinate, but the union may fail to match the combined scope before the frontier. One change lifts it. Running the three dumps against a single shared engine snapshot whose coordinate is recorded makes that exhibited coordinate a witness for all three units, so the dumped scope regains T3 while t_4 's chunks still mix in at T2 for the union. The choice that changed the level was the witness structure and nothing else. Dump versus scan, serial versus parallel, and bracket width do not alter the theorem once their observation and equivalence conditions hold.

5.4 T0 in the wild

The polling class shows the boundary is real. A connector that periodically issues `SELECT...WHERE updated_at > t` and emits the returned rows consumes no commit-ordered log, because its only signal is each row's own last-modified stamp. Deletes are invisible, since a vanished row simply stops appearing and no event ever reports its absence. S-LOG

has nothing to promise over. Nor do the poll timestamps admit a faithful prefix reading. The stamps are assigned by clients or by statement time rather than at commit, so rows become visible out of stamp order and the connector's own membership decision, "stamp greater than my last poll," disagrees with every mapping to commit-order prefixes. Both failures are independent, either alone is disqualifying, and neither is repairable by widening any bracket. The class is T0, lying outside the family rather than being a weak member of it.

6 Merge Semantics

The canonical rule of §3.4 defines the meaning of an emission. An implementation may emit a different stream, as both buffering disciplines do. To use the cut theorem, the implementation must show that replaying its stream yields the same sink_f as the canonical rule. This section proves that equivalence for the disciplines used by the five instances. It also states, without claiming proofs, the obligations for four further disciplines. The withholding rule of §6.3 is one correctness condition that becomes visible only at this layer. Throughout the section we fix a source, a coordinate space, and a conforming capture plan at frontier f . Each discipline adds its own consumption start and emission clauses.

6.1 Streams and their replay

Definition 6.1 (Stream replay). An *emitted stream* is a finite sequence of entries, each carrying a key and a value-or-absence result. Log events serve as entries directly. Its *replay* is the per-key fold from the empty state. A key's last entry decides its value, and a key with no entry replays to absent.

Corollary 4 (Actual emitted replay at the frontier). *Let a source, coordinate space, and capture plan at frontier f satisfy the contract. Let E be a finite emitted stream. If*

$$\text{replay}(E)(k) = \text{sink}_f(k) \quad \text{for every } k \in K,$$

then

$$\text{replay}(E)(k) = \sigma_{\llbracket f \rrbracket}(k) \quad \text{for every } k \in K.$$

PROOF. For each scoped key, substitute the assumed emission equivalence into Theorem 1. $\square$

Corollary 4 is the reusable bridge from a modeled or actual finite output to the source-side result. It states final replay from an empty state. It does not identify physical output prefixes with Definition 4.1, prove transport or delivery, or explain application to a populated sink. Reading its conclusion as a transactional snapshot additionally requires the frontier to be transaction-closed.

Starting from the empty state is what lets absence by omission mean absence. A physical scan emits only the finitely

many present rows it returns. Its complete range membership induces implicit absence for every other identity in the unit, even when the logical domain itself is not finite. A discipline therefore never emits refresh entries for keys read as absent, whereas delete events are emitted because they arrive from the log. Both routes represent the same state, and the equivalence proofs below rely on that identification. A repair applied to a populated sink cannot use omission this way. It must clear the repaired scope before replay or materialize source absence as explicit tombstones. Sink-side execution remains outside the theorem.

One finiteness convention accompanies it. Each unit has finitely many rows to emit at its close, as every real chunk, a finite set of physical rows, does. Close blocks are therefore finite, and every emission below is the finite sequence Definition 6.1 requires. Beyond its finite family of units, the contract layer imposes no finiteness requirement on unit domains or refresh maps. That additional condition is needed only at the stream layer.

6.2 Window-discard

The classic discipline buffers a unit's refresh and applies the rule that the log wins. On any in-window event for a buffered key, the buffer entry is discarded and the event flows through. When the window closes, the still-buffered rows are emitted. The keys that survive to the close are exactly those with no event in the window.

Definition 6.2 (Survivors and window-discard emission). For a unit i , the *survivors* are the keys $k \in D_i$ with $R_i(k) \neq \perp$ and no event in $(lo_i, hi_i]$. Fix a consumption start s_0 with $s_0 \sqsubseteq lo_i$ for every unit. The *window-discard emission* is one pass over the consumed events $(s_0, f]$ in commit order, emitting every event at its own place. Immediately before the first consumed event beyond a unit's high edge, or at the end of the pass if none exists, the unit *closes*. One refresh entry per survivor is emitted, carrying the read result. Distinct units never share a survivor key, by O1, so the order of same-position close blocks is immaterial.

Lemma 6.1 (Per-key shape of the discard emission). *Let $k \in D_i$. The entries for k in the window-discard emission are: k 's consumed events in $(s_0, hi_i]$, in commit order. Then, exactly if k is a survivor of unit i , one refresh entry with result $R_i(k)$. Then k 's events in $(hi_i, f]$, in commit order.*

PROOF. Events for k pass through at their own places, and the pass visits them in commit order. Only unit i can emit a refresh entry for k , because a survivor entry for k presupposes $k \in D_j$ for the emitting unit j , and O1 gives $j = i$. That entry, when it exists, is emitted at unit i 's close, which sits after every consumed event at or before hi_i and before every consumed event beyond hi_i . $\square$

Theorem 2 (Window-discard equivalence). *For every scoped key, the replay of the window-discard emission equals the canonical replay: $\text{replay}(\text{emission})(k) = \text{sink}_f(k)$ for all $k \in K$.*

PROOF. Fix $k \in D_i$ and write $W = (lo_i, f]$ for the canonical window. Lemma 6.1 lists k 's stream entries. We compare last entries against the canonical fold, splitting on where k 's events fall.

Some event for k lies in $(hi_i, f]$. The last such event is k 's last stream entry, since post-close events follow everything else. It is also the last event of W , because W consists of $(lo_i, hi_i]$ followed by $(hi_i, f]$. Both replays return its image.

No post-close event, but some event for k lies in $(lo_i, hi_i]$. Then k is not a survivor, so its stream entries are exactly its consumed events in $(s_0, hi_i]$, and the last of these is the last event of $(lo_i, hi_i]$, which in turn is the last event of W . Again both replays return its image.

No event for k in W at all. The canonical side returns $\text{sink}_f(k) = R_i(k)$, and by O2, validity at some in-bracket c together with Lemma 4.1 gives $R_i(k) = \sigma_{\llbracket lo_i \rrbracket}(k)$. Two sub-cases on the read result. If $R_i(k) \neq \perp$, then k is a survivor. Its refresh entry is its last stream entry and the stream replays to $R_i(k)$ directly. If $R_i(k) = \perp$, then k is not a survivor and its stream entries are its consumed events in $(s_0, lo_i]$, of which there may be none. With none, the stream replays k to absent, which equals $R_i(k)$. Otherwise let q be the last such event. q is then the last event for k in $\llbracket lo_i \rrbracket$, so $\text{img}(q) = \sigma_{\llbracket lo_i \rrbracket}(k) = R_i(k) = \perp$. The entry is a delete, and the stream replays k to absent once more. In every sub-case the stream replay equals $R_i(k) = \text{sink}_f(k)$. $\square$

The stream may carry pre-bracket events for a key that the canonical rule never consults, because consumption started before the bracket did. Condition O2 ensures that those events are consistent with the read result. A key read as absent can reach the sink either by never being mentioned or by ending on its own delete, and the two routes agree. Symmetrically, a surviving refresh entry is emitted *after* any pre-bracket event for its key and overrides it, which is the ordering Definition 6.2 builds in by closing units inside the pass rather than up front.

Theorem 3 (Window-discard monotonicity). *Tag each emitted entry with a count of committed events, tagging each event with the length of the log prefix ending at it and each surviving refresh entry with the length of $\llbracket hi_i \rrbracket$, its unit's close. Then along the window-discard emission, each key's tags are non-decreasing, so no entry ever replays a key to an older state after a newer one.*

PROOF. By Lemma 6.1, k 's entries are its pre-close events, whose tags are at most the length of $\llbracket hi_i \rrbracket$ and appear in commit order. Then possibly the refresh entry, tagged exactly

that length. Then its post-close events, in commit order, with tags beyond it. Each block is internally non-decreasing and each boundary steps upward. $\square$

The surviving refresh's tag is exact by Lemma 4.1. A survivor's window contains no event for the key, so a read result valid somewhere in the bracket matches the key's state at the high edge.

6.3 Buffered range-merge

The second concrete merge discipline reads a unit's window as a ranged backfill, upserts the window's events onto the refresh buffer, and emits the merged unit once, as plain inserts. The live stream carries only events beyond the unit's high edge. Two clauses make the discipline lawful, and both are necessary.

Definition 6.3 (Merged result). For a unit i and $k \in D_i$, the merged result is

$$M_i(k) = \begin{cases} \text{img}(e^*) & \text{if } e^*, \text{ the last event for } k \text{ in } (lo_i, hi_i], \text{ exists,} \\ R_i(k) & \text{otherwise:} \end{cases}$$

the buffer after the upsert, where the key's last in-window event decides and a key with no in-window event keeps its read result. A delete-then-reinsert key is present again, because only the last event counts.

Lemma 6.2 (The merged buffer is the state at the high edge). For every unit i and every $k \in D_i$, $M_i(k) = \sigma_{\llbracket hi_i \rrbracket}(k)$.

PROOF. If some event for k lies in $(lo_i, hi_i]$, the last one is also the last event for k in $\llbracket hi_i \rrbracket$, by the split of $\llbracket hi_i \rrbracket$ at lo_i , and both sides are its image. Otherwise $M_i(k) = R_i(k)$, and O2 with Lemma 4.1 carries the in-bracket read value to the high edge: $R_i(k) = \sigma_{\llbracket hi_i \rrbracket}(k)$. $\square$

This makes the discipline's clause (b) exact. The merged unit represents each key's value or absence at the high edge. Keys absent at the high edge are emitted zero times, which under Definition 6.1 replays as absence.

Definition 6.4 (Gated range-merge emission). Fix a consumption start s_0 with $s_0 \sqsubseteq hi_i$ for every unit. The *range-merge emission* is one pass over the consumed events $(s_0, f]$ in commit order, subject to the *withholding gate*, clause (a), under which an event whose key belongs to unit i passes to the stream only beyond unit i 's high edge, outside $\llbracket hi_i \rrbracket$. Events at or before the high edge for the unit's keys are withheld and never emitted individually. At unit i 's close, placed as in Definition 6.2, one entry per key with $M_i(k) \neq \perp$ is emitted, carrying $M_i(k)$. The merged block is the unit's sole emission at or before its high edge.

The observability requirement is genuinely weaker here than for window-discard, and the difference matters for the parallel instance. Window-discard consumes one pass from $s_0 \sqsubseteq lo_i$. Range-merge reads each window by a coordinate-addressable backfill (the S-OBS retention clause of §3.2) and needs the *stream* only from $s_0 \sqsubseteq hi_i$. Everything a unit's keys would contribute at or before the high edge is withheld by clause (a) anyway, and the window's content enters through the buffer instead. That weaker requirement is precisely the handoff condition that the parallel-chunks instance must name as an assumption (§7).

Lemma 6.3 (Per-key shape of the gated emission). Let $k \in D_i$. The entries for k in the range-merge emission are: if $M_i(k) \neq \perp$, one merged entry with result $M_i(k)$ at unit i 's close. Then k 's events in $(hi_i, f]$, in commit order. Nothing else is emitted, because every earlier occurrence of k is withheld.

PROOF. The gate blocks k 's events at or before hi_i from the stream, and O1 makes unit i the only unit whose close can mention k . Post-close events pass the gate, at their own places and in commit order, after the close block. $\square$

Theorem 4 (Range-merge equivalence). Under clauses (a) and (b), the replay of the range-merge emission equals the canonical replay: $\text{replay}(\text{emission})(k) = \text{sink}_f(k)$ for all $k \in K$.

PROOF. Fix $k \in D_i$. If some event for k lies in $(hi_i, f]$, its last such event is the last stream entry for k and also the last event of $(lo_i, f]$, so both replays return its image, as in Theorem 2. Otherwise no event for k lies beyond the high edge, and the canonical fold collapses to the merged result. If $(lo_i, hi_i]$ holds an event for k , its last one decides both $\text{sink}_f(k)$ and $M_i(k)$. If not, both equal $R_i(k)$. Hence $\text{sink}_f(k) = M_i(k)$. On the stream side, Lemma 6.3 leaves either the single merged entry, replaying to $M_i(k)$, or, when $M_i(k) = \perp$, no entry at all, replaying to absent, which is $M_i(k)$ once more. $\square$

Theorem 5 (Range-merge monotonicity). With tags as in Theorem 3 and the merged entry tagged with the length of $\llbracket hi_i \rrbracket$, each key's tags are non-decreasing along the range-merge emission.

PROOF. By Lemma 6.3 a key's entries are one merged entry at its unit's close, accurately tagged by Lemma 6.2, followed by events with tags beyond it, in commit order. $\square$

Clause (a)'s full reach is necessary. A natural implementation deduplicates exactly what it buffered by absorbing the window's events into the merged unit while streaming every other consumed event, narrowing withholding to the window alone. That variant is wrong, and the failure needs only two events.

Proposition 3 (Withholding is necessary). *There is a contract instance on one key for which the gated emission replays correctly, while the window-narrowed variant (absorb the window into the buffer, stream every other consumed event) replays the key to a value the source no longer holds.*

PROOF. Over an initially empty store, take $L = e_1 e_2$ with $\text{key}(e_1) = a$, $\text{img}(e_1) = 5$, $\text{key}(e_2) = a$, $\text{img}(e_2) = \perp$, representing an insert followed by a delete. Coordinates c_0, c_1, c_2 map to the prefixes of lengths 0, 1, 2. The plan has one unit with domain $\{a\}$, bracket (c_1, c_2) , frontier $f = c_2$, and refresh $R(a) = 5$, reflecting the state at the low edge c_1 where the insert has committed and the delete has not. Consumption starts at $s_0 = c_0$.

The window $(c_1, c_2]$ contains the delete, so the merged result is $M(a) = \perp$ and the lawful emission for a is empty. By Theorem 4 it replays a to absent, which is $\sigma_{\llbracket c_2 \rrbracket}(a)$. Under the window-narrowed variant, the in-window delete is absorbed into the buffer, whose merged block is empty and emits nothing, while the pre-bracket insert e_1 lies outside the window and passes to the stream. The variant's stream is the single entry e_1 , and it replays a to 5, which is a value the source deleted before the frontier. Clause (a)'s full reach, withholding everything at or before the high edge rather than only what the buffer absorbed, is exactly what separates the correct replay from the wrong one. $\square$

Window-discard and range-merge ship visibly different streams from the same plan. The first interleaves a unit's in-window events at their own places, whereas the second absorbs them into one merged block. Theorems 2 and 4 say both replay to the same canonical sink. The log-wins property is proved once for the canonical rule, and each concrete merge discipline then needs only a short equivalence proof.

6.4 Point-splice

A point bracket has $lo_i = hi_i$, the window is empty, and there is nothing to reconcile. No key has an in-window event, and the merged result is the refresh itself. A point splice is the degenerate case of window-discard, with $s_0 \sqsubseteq lo_i$ for every unit. Each unit emits its present refresh entries at its splice coordinate, followed by later events in log order. A composite with distinct splice coordinates must therefore begin at or before the earliest splice, because beginning at the latest would omit events needed by earlier units. Theorem 2 gives replay equivalence, and Corollary 4 gives final source equality. This is equality after replay, not entry-for-entry identity. The splice instances of §7 use this degenerate case.

6.5 Analyzed and open

Four further disciplines run in production or arise naturally from the model. For each we state the equivalence obligation in the contract's terms and record it as *open*. None is established in this paper, and nothing in this subsection is a result.

Readers evaluating only the five production placements may continue directly to §7. This subsection records design obligations for additional mechanisms.

Key-range gating. This discipline uses no buffer at all. The capture maintains a copy cursor over a totally ordered key domain, applies log events only for keys at or below the cursor, and discards events for keys ahead of it, on the grounds that the later copy pass will read those rows anyway. The obligation has two parts. The plan must be a monotone cursor over the ordered domain, and each key's refresh witness (O_2 's coordinate) must lie at or after every event discarded for that key while it was ahead of the cursor, so that the eventual read is fresh under an advancing view. A frozen read view breaks the second part. Discarding an observed ahead-of-cursor event and then fetching that row from an older snapshot loses the discarded change with nothing left to repair it. A non-monotone visit order breaks the first. Both preconditions are stated here. Neither implication is proved, and the discipline is open.

Version-max. With a version column on every row, a sink can apply refresh entries and events in any order and keep, per key, the entry of maximal version, allowing reconciliation to succeed without any bracket. Deletes would also need a versioned tombstone or equivalent evidence to reject stale copied rows. The obligation is that the version order agree with commit order on every pair of entries for the same key. Engine-generated, commit-ordered versions can satisfy it, whereas client-supplied timestamps in general do not, and a deployment leaning on them sits at T1 (§5) until the agreement is established, so the discipline remains open.

Re-read normalization. Under a delta log, events carry changed columns rather than full images, S-IMG fails, and the core fold is not even well typed. The industrial repair is to re-select colliding keys under a fresh bracket and emit the read images. In this model the repair is a *log normalization*, not an in-core discipline. The repaired instance is modeled over effective full-image events, each re-read supplying the image its delta event lacked, and the instance's equivalence obligation then includes the normalization itself. That obligation has a termination half, since re-reads race new deltas and the re-read procedure must be shown to terminate. None of the vendor documentation surveyed for this paper states one. The discipline remains open, with the termination half flagged as the substantive part.

Rewind-by-preimage. Where the log carries before-images, a late read can be rolled backward by applying preimages in reverse until the unit’s low edge, yielding a refresh valid at lo_i itself. Its obligation is to identify the suffix reflected in the read and show that reversing it reconstructs each key at lo_i . It is recorded as a construction worth having, but is entirely unproved and remains open.

7 Instantiating the Contract

The preceding sections separate three questions. Does a capture plan satisfy the source-side contract? Does its emitted stream replay like the canonical merge? Which semantic level and certification status follow? This section answers those questions for five production protocol shapes. It is the formal instantiation layer of the paper. Section 8 distills the resulting framework and its operational consequences into a standalone practitioner synthesis.

7.1 How a protocol inherits the theorem

Each instantiation follows the same proof pattern.

- (1) Map the protocol’s coordinates to nested log prefixes consistently with its actual coordinate behavior, including the exact interpretation of its low and high edges.
- (2) Map its copied scopes, reads, brackets, and frontier to a capture plan, then satisfy the source promises and plan obligations of Definition 3.2.
- (3) Match the emitted stream to a proved discipline in §6, or state the additional equivalence condition that the implementation must establish.
- (4) Apply Theorem 1 for the frontier result and Lemma 4.2 for the conservative post-close trajectory. Apply Corollary 1 only when the capture exhibits one shared state witness. Add transaction closure before reading either result as a transactional snapshot or history.

The proof is conditional at the points where the protocol description does not itself establish a source, read, coordinate, or emission property. Such requirements appear below as explicit preconditions and theorem assumptions. Release-specific evidence establishes whether a particular implementation satisfies them.

7.2 What an instantiation establishes

A contract proof establishes the canonical frontier result. An emitted stream attains it only after the corresponding merge-equivalence argument is established. The two claims are kept separate below because the same plan can be emitted by different buffering and handoff rules. Likewise, a T3 placement concerns the retrospective canonical family of Definition 4.1 and an exhibited common state witness. It does not assert

that every physical prefix of an emitted stream follows that family. Those distinctions let the formal placements remain precise while the practitioner synthesis can focus on what the framework means and when its results apply.

7.3 Classic watermarked DBLog

Construction. The original algorithm [1] reads each table in chunks. It brackets every chunk select between two watermark writes, updates to a dedicated single-row table whose change events appear in the tailed log. It takes the log positions of those two events as the bracket and deduplicates by window-discard (§6.2). Its published formalization [6] defines wellformed runs of this shape and proves them correct. That the present framework contains the original as an instance is therefore a theorem with a precise form. Every wellformed run yields a contract instance, and the contract’s replay agrees with the published formalization’s own conclusion.

The import adopts five clauses of published wellformedness, restated here in this paper’s vocabulary:

- *source binding:* the run’s source history is a committed, commit-ordered event history with coordinates non-decreasing along it.
- *chunk partition:* the chunks’ key domains are pairwise disjoint and jointly cover the run’s scope.
- *chunk evidence:* each chunk carries one read coordinate lying between its lower and upper watermarks, and its read map is total on its domain.
- *read validity:* each chunk’s read results equal the source’s row-event replay state at that chunk’s read coordinate, key by key.
- *frontier bound:* every read coordinate lies at or before the run’s frontier.

Theorem 6 (Classic DBLog satisfies the contract). *Every wellformed run of the classic algorithm, in the sense of [6], determines (i) a contract instance (Definition 3.2) and (ii) an instance whose canonical replay at the run’s frontier equals the run’s own source row-event replay state there, which is, per key, the image of the last history event for the key with coordinate at or before the frontier, or the initial value if no such event exists.*

PROOF. Map the run’s objects onto the contract’s. The source history maps to the log. Inserts and updates become events with present images, while deletes become events with absent images. This is the 2020 all-columns post-image assumption serving as S-IMG, and S-LOG is the source binding. A history coordinate identifies the prefix of all events with coordinates at or before it. Since coordinates are non-decreasing along the history, that filter is a genuine prefix,

and increasing a raw coordinate either leaves the corresponding log prefix unchanged or extends it. Different raw coordinates can refer to the same prefix. This at-or-before convention is the instance’s declared endpoint normalization (§2.3). The prefix includes every event at the chosen coordinate, so a run of equal-coordinate events lands wholly inside it, matching the published formalization’s own latest-lookup convention, which resolves coordinate ties by position.

Each chunk becomes a unit. Its domain is the chunk domain, its bracket edges are the lower watermark and the upper watermark truncated at the frontier, and its refresh is the chunk’s read map. The truncation is needed because the published wellformedness bounds the *read* coordinate by the frontier while never bounding the upper watermark, whereas contract plans live at or before the frontier by definition. Chunk evidence and the frontier bound keep the read coordinate inside the truncated bracket, so nothing is lost. The obligations are now satisfied clause by clause. O1 is the chunk partition. O2 holds with every key of a chunk witnessed uniformly at the chunk’s one read coordinate, whose validity is the read-validity clause. This is the upward direction of Proposition 1’s discussion, a single read point instantiating the per-key existential. O3 is the totality half of chunk evidence. O5 holds by construction, since the import maps the same history that generated the change stream, so both paths share one event vocabulary. This proves (i).

For (ii), the prefix for a coordinate consists exactly of the mapped events with coordinates at or before that coordinate, so the state after it is, key by key, the image of the key’s last such event (ties across equal coordinates resolved by position, per the at-or-before reading declared above), or the initial value if none exists. Theorem 1 applied to the instance equates the canonical replay at the frontier with exactly that state. $\square$

The published formalization’s observed-stream clauses and its replay and certificate layers are not needed here. In this framework the log is the committed history itself, and observation enters at the discipline layer through S-OBS (§6). The two developments meet at their conclusions. The state in Theorem 6(ii) is precisely the state the published formalization’s own replay theorem computes at the frontier, the same per-key latest lookup under the same position-resolved tie convention [6]. The contract-level theorem thereby generalizes the published result and never contradicts it.

Theorem 6 imports the plan-level T2 result into the generalized framework. The earlier formalization separately proves a clean-prefix theorem for the emitted stream in its own run semantics [6]. A physical generalized window-discard conclusion needs an additional bridge. It requires S-OBS, the survivor and close behavior of Theorem 2, and every real upper marker at or before the frontier being claimed.

The last condition matters because the import replaces an upper marker beyond f by f in the canonical plan. A formal close at that truncated edge is not automatically a physical close in the original run. One may instead choose a later frontier that includes the real marker. This is a statement about where the proof stops. It says nothing about the correctness of classic DBLog code. Corollary 4 gives the final emitted equality once the bridge holds. None of these results claims that each prefix of the emitted stream follows the canonical trajectory of Definition 4.1.

Watermark preconditions (W1-W5). The formal import assumes wellformedness. A deployment must also be *able* to run the bracket mechanism. The watermark-write design carries five preconditions, made explicit here because §9 shows the 2020 paper’s own support conditions never state them:

- W1** *provisioning*: the watermark table exists in a dedicated namespace, or the capture role holds DDL privilege to create it.
- W2** *write access*: the capture role can update it, twice per chunk, for the life of every run. Read-only sources, read-only roles, and frozen archives are excluded.
- W3** *capture coverage*: the watermark table’s changes appear in the same tailed stream as the data, a configuration property of publications and log filters. A filtered-out low watermark prevents the protocol from observing the window opening, and without an external timeout or recovery mechanism the run can wait indefinitely.
- W4** *recognizability*: the watermark value column is among the captured columns, so any column filtering must retain that value.
- W5** *ordering coupling*: watermark writes, chunk reads, and the tailed log target the same instance, one linear commit history. Using a primary for writes and a replica for reads requires a separate argument that the read lies within the watermark bracket in the same history.

Result. The imported plan is T2, by Theorem 6 and Theorem 1. A frontier-replay result for the emitted stream additionally depends on the physical-marker, observation, and window-discard requirements stated above. The certified configuration of the published formalization additionally carries an externally checkable certificate of its cut, which is the T4 certification overlay (§5.1). The classic instance is where the framework incorporates its T4 anchor [6].

Debezium’s signal-table modes. In the documented non-read-only mode, Debezium’s default `insert_insert` strategy writes opening and closing records to the configured signaling data collection and reconciles the intervening row

events by window-discard [4,13]. This has the same span-bracket and log-wins shape as the classic placement, under four conditions. W1-W5 hold for the signaling collection. The opening record precedes the chunk read, and the closing record follows it. The scan returns one value-or-absence result for every owned key, each valid at some coordinate inside the bracket (O2-O3). Release waits until the real close and all relevant window events have been observed. Stable scope, identity, and representation agreement remain required.

Under those conditions, Theorem 1 places the plan at T2. Theorem 2 and Corollary 4 give the finite emitted output final source replay. This bridges Debezium’s documented mode to the watermarked placement under the stated conditions. It does not apply Theorem 6’s imported wellformedness to a Debezium release.

The insert_delete strategy uses the same bracket. The opening insert supplies the low edge, and deleting that row supplies the high edge. In the tagged source, Debezium accepts the delete as a signaling event, reads the opening row’s identifier from its before-image, and converts it to the same internal window-close signal used by insert_insert [18]. This placement therefore adds the requirement that the delete event’s before-image retain the opening row’s identifier (the signal table’s primary key). Without it, the connector cannot recognize the close. Enabling the source signaling channel also does not itself establish capture coverage. The signaling collection’s changes must actually be present and recognizable in the consumed history.

7.4 Debezium read-only capture: MySQL and MariaDB

Coordinate construction. Debezium’s incremental snapshots adapt DBLog’s watermark-based design [3]. Its MySQL and MariaDB read-only connectors bracket chunks by reading server GTID state instead of writing marker rows [4,14,19]. The trigger can also arrive over a write-free channel [13]. The two servers do not expose the same coordinate, however. MySQL reports a full executed-GTID set. MariaDB’s GTID_BINLOG_POS reports one frontier per replication domain. We therefore prove the set-oracle construction once, then give separate product bridges below.

The generic construction applies to a coordinate that is the exact set of transactions contained in one transaction-complete prefix of the consumed history. The difference between high and low then selects the complete transaction blocks that entered the prefix while the chunk was being captured. This is stronger and more precise than calling the sample atomic. A sample must end between complete transaction blocks and be gap-free relative to that same consumed history.

Model the history as follows. Positions p of the log carry a transaction label $\tau(p)$, and one assumption shapes the labeling. Equal labels occupy contiguous position blocks, because a transaction commits as one uninterrupted run of row events. Call a position n a *boundary* when no transaction straddles it, meaning no position before n shares a label with any position at or after n . After projection onto labels represented in the scoped log, every accepted sample must equal the complete transaction-label set of the prefix at one such boundary. The *executed set* at boundary n is

$$G(n) = \{ \tau(p) : p < n \}.$$

An operational executed set can also contain transactions that have no row event in the capture scope. The model uses the projection of that set onto transaction labels represented in the scoped log. Applying the result therefore requires projection agreement. For every captured event, membership of its label in the raw low and high sets must agree with membership in the projected sets $G(n)$. Extra labels are harmless because the window test asks only about the captured event’s own label. This bridge is included in A-linear below.

The set $G(n)$ maps to the length- n prefix. The mapping is well defined because distinct boundaries carry distinct executed sets. If $m < n$ are boundaries, the label $\tau(m)$ lies in $G(n)$ but not in $G(m)$, since membership in $G(m)$ would put an earlier position and position m on one label across the boundary m . The corresponding prefixes are therefore nested, and inclusion between executed sets agrees with prefix inclusion, so Definition 2.1 is satisfied without any scalar.

What remains is admissibility’s fidelity requirement (§2.2), under which the instance’s own membership decision must agree with the window defined by the mapped prefixes. That is a theorem.

Theorem 7 (Transaction-complete executed-set windows are faithful). *Let $n_{lo} \leq n_{hi}$ be boundaries and p any log position. Then*

$$\tau(p) \in G(n_{hi}) \setminus G(n_{lo}) \iff n_{lo} \leq p < n_{hi}.$$

The operational test, “the event’s transaction identifier lies in the high set and not in the low set,” decides exactly the half-open window of log events.

PROOF. Suppose $\tau(p) \in G(n_{hi}) \setminus G(n_{lo})$. If p lay below n_{lo} , its label would be in $G(n_{lo})$ by definition, so $n_{lo} \leq p$. If p lay at or beyond n_{hi} , then membership in $G(n_{hi})$ would give an earlier position $q < n_{hi}$ with $\tau(q) = \tau(p)$, and q and p would straddle the boundary n_{hi} . Hence $p < n_{hi}$. Conversely let $n_{lo} \leq p < n_{hi}$. Then $\tau(p) \in G(n_{hi})$ directly, and $\tau(p) \notin G(n_{lo})$ because an earlier position $q < n_{lo}$ with the same label would straddle the boundary n_{lo} . $\square$

Theorem 7 establishes the coordinate-fidelity requirement of the read-only placement. The remaining work is to check the common contract and the buffer-and-discard emission.

Proposition 4 (Placement: read-only). *Consider a plan whose units carry exact transaction-complete executed-set brackets, whose final domains partition the scope, and whose high boundaries lie at or below the frontier. Assume A-linear, A-scope, A-read, and A-representation as stated below. Then:*

- (1) *the plan satisfies the contract, and its canonical replay is T2 by Theorem 1*
- (2) *a finite buffer-and-discard output has the same final source-side replay under A-discard and the window-discard form of S-OBS, by Theorem 2 and Corollary 4.*

PROOF. Theorem 7 gives coordinate fidelity. A-scope gives O1 and O3. A-read gives O2. A-representation gives S-IMG and O5, while A-linear binds every object to S-LOG. The bracket and frontier bounds complete the contract, so Theorem 1 proves part 1. Under A-discard and S-OBS, the emitted stream realizes the window-discard construction. Theorem 2 proves emission equivalence, and Corollary 4 composes it with the cut for part 2. $\square$

Read-only assumptions.

A-linear *history, prefix, and projection:* samples, chunk reads, and streamed events refer to one unchanged commit-ordered history whose transaction identifiers occupy contiguous blocks. Each accepted sample, after the declared projection, equals the gap-free set of complete transaction blocks in one prefix of that history. For every captured row event, membership of its transaction identifier in the raw sample agrees with membership after projection to the modeled scope. These are three separate clauses. One cannot be inferred from either of the others.

A-scope *complete stable ownership:* the final unit domains partition the captured identity namespace and remain stable for the run. The scope witness is coordinate-bound. Each chunk scan supplies complete membership, so a nonreturned owned identity is a justified absence rather than an unobserved row.

A-read *per-key bracket match:* O2 at instance level. Every returned value or absence must match the source at some transaction-boundary prefix inside the bracket. This condition is kept explicit because the connector documentation does not fully specify how chunk reads account for every key and ensure that each returned value matches source state within the sampled interval [4,14].

A-representation *image and identity agreement:* the streamed and copied paths share the same stable key identity and value encoding. Effective events carry complete post-images, deletes represent absence, and a primary-key change is normalized as old-key absence plus new-key value. This satisfies S-IMG and O5.

A-discard *serialized window lifecycle:* the low sample is installed before the read and the high sample is taken after the buffer is complete, or an equivalent serialization establishes that order. Every relevant row event whose transaction lies in the high-minus-low window can remove its buffered key. The buffer is released only after all row events of the last window transaction have been processed. The first later transaction may establish closure before processing its own row events because those events lie outside the window. A heartbeat may close only after all window events are known to have been processed.

S-OBS *complete advancement and coverage:* one commit-ordered consumed stream begins at or before every low boundary and continues through the frontier. Filtered and out-of-scope events needed to advance the window state remain observable even when they do not enter the emitted data stream. Retention covers every range the discipline reads.

MySQL bridge. Debezium 3.6.1.Final samples MySQL's Executed_Gtid_Set through binary-log status before and after a chunk read, subtracts the low set from the high set, and uses event membership to manage the in-memory window [4,20]. This is the direct full-set bridge to Theorem 7 under A-linear. MySQL documents that a committed GTID is externalized into `gtid_executed` non-atomically shortly after commit [21]. The bridge therefore does not assume atomic externalization. It assumes the stronger semantic fact the proof needs, namely that each accepted value maps to an exact complete prefix of the same history used by the read and consumed log. GTID mode and GTID consistency are required. On a multithreaded MySQL replica, the documented commit-order-preservation setting is also needed for this one-history mapping [4,21].

MariaDB bridge. Debezium 3.6.1.Final instead samples GTID_BINLOG_POS. MariaDB defines this value as the last event group written to the binary log in each replication domain, so it is a vector of per-domain frontiers rather than MySQL's complete executed set [22]. The tagged connector samples that value. A nonempty high-minus-low GTID difference causes the shared binlog source to request a reread. The common reread helper also requires a running snapshot,

an open deduplication window, and a nonempty buffer, so the request alone does not establish an effective retry [20,23]. Under the stated assumptions, we conclude correctness in two cases. If the high-minus-low difference is empty, the same-history and monotone-prefix assumptions make the complete vector a confirmed quiet point in the exact consumed MariaDB binlog. The chunk is then a point splice, provided A-read supplies complete value-or-absence results and the vector maps consistently to the same prefix of the consumed log across every relevant domain.

If the difference is nonempty and the buffer is empty, the reread helper returns without another scan. This case needs no quiet-point argument. Under A-read and A-scope, every refresh is a justified absence, so the survivor set is empty. The same-history and monotone-prefix assumptions still ensure that the two samples define a well-formed bracket. Under A-discard and S-OBS, every racing or later event, including an insert that raced the empty scan, remains on the emitted log path. Theorem 2 therefore gives final source equality. This argument uses the samples as bracket bounds. It does not invoke the MySQL executed-set membership theorem or MySQL-only replica variables. These two cases do not exhaust the tagged implementation. For a populated buffer with a nonempty difference, we would need to show separately that the sampled coordinates identify the correct interval around the read in the committed log and that replaying the connector’s output reconstructs the source state. Alternatively, we would need evidence that a reread actually occurs and brings the chunk into one of the covered cases. MariaDB deployments with independently advancing domains must state the same-history and prefix-preserving translation explicitly [24].

Heartbeat availability remains an operational liveness requirement for detecting that the stream advanced. It does not replace A-linear, S-OBS, or the close ordering in A-discard. The PostgreSQL read-only variant brackets with transaction identifiers rather than executed sets. Its documentation frames the watermarks through the current in-progress transaction [11], and the exact low and high construction is not documented in a form this proof can model. We do not translate that phrase into access to in-progress events or dirty state, as both are excluded by the model. We therefore do not place that variant. The boundary lies in the available documentation.

Result. Under their respective bridges and the shared assumptions, the MySQL set plan and the two analyzed MariaDB cases are T2. The empty-difference case uses a point splice, while the empty-buffer, nonempty-difference case uses window-discard over the sampled bracket. Their actual finite outputs have the same final source replay only under the lifecycle, observation, and emission-equivalence

conditions of the corresponding discipline. Write-freeness is visible in both constructions because none of Proposition 4’s assumptions places a marker event in the log.

7.5 Flink CDC parallel chunks

Protocol mapping. The protocol splits a table into snapshot chunks that may run in parallel. The release-3.6.0 source records a low binlog offset, runs the chunk select, records a high offset, reads the corresponding binlog range, and reconciles those events into the buffered chunk before hand-off [5,25]. This is the range-merge discipline of §6.3. The plan-level mapping and the properties of the stream actually emitted by the connector are explicitly separated below.

Offsets map to prefixes of one source history. The documented $[LOW, HIGH]$ notation does not by itself fix the raw endpoint behavior. A-edge therefore maps the connector’s raw offset tests to the half-open window of log events $[[HIGH]] \setminus [[LOW]]$. Comparisons and membership tests must agree with that mapping.

Proposition 5 (Placement: parallel chunks). *Consider a finite final plan of logical chunk units whose offset brackets have high edges at or below the frontier. Under A-source, A-read, A-identity, A-edge, A-normalize, and A-representation:*

- (1) *if the final logical domains partition the captured scope, the plan satisfies the contract and its canonical replay is T2 by Theorem 1*
- (2) *if A-window, A-merge, S-OBS, and A-handoff also hold, the finite emitted stream has final source-side replay equality by Theorem 4 and Corollary 4*
- (3) *if the actual normalized and gated per-key sequence is the sequence modeled by Definition 6.4, its source tags are monotone by Theorem 5.*

PROOF. For part 1, the explicit partition hypothesis gives O1. A-read gives O2 and O3. A-identity supplies stable logical ownership and final assignment. A-edge supplies bracket order and coordinate fidelity. A-source, A-normalize, and A-representation supply the standing history, image, and representation conditions, including O5. The plan-level proof requires no handoff condition. For part 2, A-window and A-merge identify the finite output with the gated range-merge construction, S-OBS supplies the needed log coverage, and A-handoff starts the stream at or before every high edge and supplies clause (a). Theorem 4 gives canonical replay equivalence, which Corollary 4 lifts to source equality. Part 3 is Theorem 5 applied to that actual normalized sequence. $\square$

Independence from bracket ordering. The plan proof relates distinct units through final logical ownership and one shared frontier bound. It does not order one chunk’s bracket against another’s, so brackets may overlap or nest. This is narrower than saying that the implementation needs no coordination.

Split assignment, checkpointing, phase transition, per-split event filtering, and recovery still coordinate the running connector. The emitted-stream result also needs the global handoff and gate. What the theorem removes is only an additional pairwise bracket-order assumption. A conforming plan over a disjoint scope composes by Corollary 3.

Parallel-chunk assumptions.

A-source *one history and scope*: offset samples, chunk reads, backfill records, and the continuing reader refer to one prefix-preserving source history and one coordinate-bound scope.

A-read *per-key bracket match*: assumed, not derived, because the chunk select’s isolation and membership guarantees are not documented here [5]. Every returned value or absence must be valid at some coordinate inside the bracket, and the scan must give a complete read result for its logical domain.

A-identity *stable logical ownership*: stable logical identities have one final logical-domain assignment for the run. A primary-key change is normalized as absence for the old key plus a value for the new key. For tables with primary keys, the tagged source uses the record key as the merge-buffer identity. Split routing and the live per-split gate instead extract the configured chunk-key value from the appropriate before or after record. A mutable non-key split column can therefore move a physical row between split predicates and is a routing hazard, so that case needs implementation-specific evidence for stable final ownership or an equivalent proof. Immutability is a useful sufficient condition where the documentation requires it, not the framework’s universal identity rule [5,25]. Flink also supports tables without a primary key when a non-null chunk column is configured. The tagged implementation then uses the row image, rather than a stable logical key, to identify buffered rows. This documented mode is not placed here unless a deployment supplies the stable-identity normalization required by §2.1. The documentation also reduces the guarantee to at-least-once if the chunk column changes [5,25].

A-edge *half-open endpoint normalization*: raw offset inclusion, comparison, and filtering implement $\llbracket HIGH \rrbracket \setminus \llbracket LOW \rrbracket$ without a gap or a double-applied boundary event.

A-normalize *row-operation normalization*: Debezium READ and CREATE records become value writes, DELETE becomes absence, and UPDATE is interpreted as its before and after pair or as its complete

after-image as appropriate. The release-3.6.0 default deserializer preserves this operation meaning in Flink RowKind values [25].

A-representation *image and encoding agreement*:

A-normalize yields S-IMG, and copied and logged paths agree on stable keys and values as required by O5. A custom deserializer must be shown equivalent. Append-only conversion is usable only when the original operation metadata is retained or otherwise interpreted before RowKind is erased. Treating every change as an independent insert does not by itself represent updates and deletes [25].

A-window *complete window availability*: for each chunk, the backfill reader can consume every scoped row event in the normalized low-to-high window. Retention covers that range.

A-merge *faithful merge realization*: the backfill updates each buffered key to the state at the high edge, including absence, and the emitted chunk represents that state.

S-OBS *complete observation*: every event needed to build and close a chunk window, including filtered events used for position advancement, remains observable through the claimed frontier.

A-handoff *live floor, gate, and order*: the continuing reader starts at or before every chunk high. For a chunk’s logical keys, events at or before its high are withheld because their effects are already in the merged result, and events after its high are released in source order. The release-3.6.0 assigner records per-split highs and starts the binlog split at their minimum, so the reader may encounter an event that precedes another split’s low edge. For each data event, the reader identifies its owning split from the chunk key and emits the event only when its position is strictly greater than that split’s high watermark [25]. It therefore withholds every event at or before the high edge, including pre-low events, as the range-merge discipline requires.

The official output contract supports the default changelog when A-normalize and A-representation hold. A custom deserializer is covered only after the same equivalence is established. An append-only mode that erases operation kind cannot be used as a value-or-absence stream unless the lost operation meaning is retained or reconstructed elsewhere. Checkpoint restart, rescaling, source failover, and end-to-end exactly-once sink application remain outside this placement. They must satisfy the plan and emission conditions to obtain the result.

Skipping the backfill. The connector offers a mode that skips the per-chunk window read entirely, deferring in-window changes to the later streaming phase. Its documentation states that skipping “might lead to data inconsistency” and that “only at-least-once semantic is promised” [5]. The documented behavior does not satisfy the requirements needed to prove merge equivalence for this mode. We therefore formulate no T2 mapping for it. The documentation’s stated at-least-once semantics remains unchanged, and this boundary is not a finding that the implementation is incorrect. A separate convergence argument could support a different result (§5.1).

Result. Under its plan conditions, every final logical chunk plan is T2 without A-handoff. Under the additional window, merge, observation, and handoff conditions, its finite output has the same final source replay. Per-key monotonicity additionally concerns the actual normalized gated sequence and does not follow from final equality alone.

7.6 Coordinate-bound reads: LSNs, binlog positions, and consistent points

Point-splice construction. Several engines bind one consistent read to one published log coordinate, collapsing the bracket to a point. Under REPEATABLE READ, MariaDB’s START TRANSACTION WITH CONSISTENT SNAPSHOT establishes an InnoDB read view whose binlog coordinate the session then reads from the status pair Binlog_snapshot_file/Binlog_snapshot_position, documented as “the binlog position that corresponds to the snapshot” and queryable “in a transactionally consistent way, irrespective of which other transactions have been committed since the snapshot was taken” [7,10,26]. Percona Server ports the same pair [8]. PostgreSQL’s logical replication slot creation exports a snapshot together with a consistent_point, “exactly the state of the database after which all changes will be included in the change stream” [9,27]. In each case a select, or a whole-table copy, runs inside that read view, and log consumption starts at the recorded coordinate. For a composite of several such units it starts at or before the earliest of their coordinates (§6.4).

All these members, and the native dumps of §7.7, instantiate *one* formal object, because the plan does not record how a read was obtained (§3.1). The unit carries a point bracket at a recorded coordinate r , with the refresh an engine-consistent read at an engine-internal view coordinate v , published as p . The instances differ in the source of their assumptions and in their operational preconditions, not in their formal plan.

Proposition 6 (Placement: the trusted point splice). *Consider the one-unit plan over dumped scope S with bracket $lo = hi = r$ and refresh R , at frontier f with $r \sqsubseteq f$. Assume*

A-engine $R(k) = \sigma_{[v]}(k)$ for every engine-backed key k : the read is consistent at the view coordinate, for keys of engines that participate in the snapshot.

A-engine-scope every key of S is engine-backed.

A-cert $v = p$: the view and the published coordinate coincide.

A-scope $r = p$: the recorded coordinate is the published one.

Then the plan satisfies the contract, and r is a shared state witness exhibited by the view-coordinate binding, so Corollary 1 applies, ensuring the canonical replay tracks the source’s row-event replay in the canonical trajectory from r to f . The instance sits at T3 on the dumped scope, with an exhibited shared-state anchor at the recorded splice coordinate. For this one point unit, r is also the latest high, so T3 adds the exact shared-state evidence rather than a longer trajectory interval. Calling that family a transactional history additionally requires transaction closure throughout the claimed range. If a finite physical output emits an exact copy of the scope before the observed suffix from r , or otherwise has the same replay, Corollary 4 also gives final source-side equality at f . This is replay equivalence, not entry-for-entry identity with a canonical stream.

PROOF. The assumptions chain into exact point equality at the recorded coordinate, such that for $k \in S$, $R(k) = \sigma_{[v]}(k) = \sigma_{[p]}(k) = \sigma_{[r]}(k)$, using A-engine with A-engine-scope, then A-cert, then A-scope. O1 holds with the single unit. O2 holds with the point witness r for every key. O3 and O5 hold at the typed refresh. The window of the point bracket is empty, so the discipline is the point splice of §6.4, and r satisfies both hypotheses of Corollary 1, because it lies in the (degenerate) bracket of the only unit and every refresh matches the source state at it. $\square$

Exact-read assumptions.

A-cert the engines’ documentation asserts the view-coordinate coincidence without exposing a mechanism to check it. O4 records the claim as an explicit assumption, not as a proved fact. Despite the assumption’s historical name, it is a trusted binding, not the independently checkable certificate that earns T4.

A-scope the recorded MariaDB status pair must identify the read view used by the copy, captured from the active snapshot session before that view is ended or replaced [10,28].

A-engine, A-engine-scope consistency is engine-scoped (§7.7 quotes the InnoDB-only caveat). The dumped scope must lie inside the participating engines’ keys.

S-OBS acquires instance-specific content. The PostgreSQL exported snapshot is valid only “until a new command is executed on this connection or the replication connection is closed” [9], so handle lifetime is a retention condition on the snapshot side, just as log retention is on the log side. Oracle documents a coordinate-bound read primitive, AS OF SCN [29], but the corresponding SCN-bound log-splice consumption side is not documented in the sources surveyed. Placing an Oracle member would require one further assumption binding the read SCN to a consumable log position, so we do not place it here.

Result. T3 on the captured scope, by Proposition 6. Per-table point splices at *distinct* recorded coordinates compose by Corollary 3 to one nonempty instance at global T2, whose conservative trajectory begins at the latest recorded splice. Proposition 2 shows that composition alone cannot promise a shared state witness or an earlier start. This is Example 5.1’s composition rule as it occurs in production. One snapshot shared across the members, its coordinate recorded, supplies the shared state witness for T3. The modeled finite copy-before-suffix output has final source replay under its exact scope and observation assumptions, with consumption starting at or before the earliest recorded splice (§6.4).

7.7 Native dumps and uncertainty intervals

Point-splice construction. The maximal chunk is an entire instance captured by the engine’s own dump tooling. `mysqldump` with `--single-transaction` `--source-data` opens a repeatable-read transaction, records the binlog coordinate at the dump’s start behind a brief global read lock, and documents the alignment as “in all cases, any action on logs happens at the exact moment of the dump” [15]. `mariadb-dump` with `--single-transaction` `--master-data` likewise records a binlog coordinate for the consistent dump [30]. `pg_dump` `--snapshot=<snapshot_name>` runs the dump inside a slot-exported snapshot, splicing at the slot’s consistent point [9,31]. No closing marker exists in any of the three. The high edge is definitionally the low edge, and “end of dump” is a duration, not a coordinate. Formally all of this is Proposition 6 again, with the dump as the one-unit refresh. What distinguishes the dump members is the basis of their assumptions, recorded here.

Native-dump assumptions.

A-cert, A-scope the native-dump member adopts Proposition 6’s two point bindings. The engine view must coincide with the tool’s published coordinate, and the coordinate recorded in the dump

must be that published coordinate. Each deployment must tie these trusted assertions to the selected tool’s documented sequence. They are assumptions, not T4 certificates.

A-engine, A-engine-scope the consistency guarantee is engine-scoped. The documentation states that “only InnoDB tables are dumped in a consistent state,” and tables of non-transactional engines dumped alongside “may still change state” [15]. A-engine-scope therefore excludes them from the dumped scope or the claim.

A-ddl the dump’s consistency prohibition on concurrent schema statements is documented and unenforced, stating that “no other connection should use ... ALTER TABLE, CREATE TABLE, DROP TABLE, RENAME TABLE, TRUNCATE TABLE” [15]. This model has a fixed key space and no schema transitions, so it represents the deployment only while the prohibition holds. There is no in-model proposition to assume, and the instance carries the precondition in this assumption list.

A-scope-witness the dump invocation or manifest enumerates the included relations and key ranges, and the dump’s recorded coordinate binds that enumeration. The coordinate can be server-wide while the dump is partial. MySQL’s emitted GTID state covers transactions “even those that changed suppressed parts of the database, or other databases on the server that were not included in a partial dump” [15]. The model therefore uses the whole-server log but restricts the claim to the enumerated dump scope.

O5 is this instance’s critical obligation. The refresh bypasses the connector’s row path entirely. Rows are parsed from dump output rather than decoded from the log, and representation agreement between the two paths, key identity and value encoding, is information a deployment must establish. The formal side is satisfied when the typed refresh is constructed. The operational side, charset, collation, and numeric edge cases agreeing between dump rendering and log rendering, is the checkable clause that O5 makes explicit.

The degradation theorem. The remaining case concerns the *untrusted* coordinate of a restored physical backup whose asserted recovery point is not certified. Without an exact view-coordinate binding the plan cannot exhibit the shared-state anchor required for T3. An interval justified by recorded bounds and containing the true view can still serve as a bracket. This placement requires the entire restored dataset in scope to equal one engine-consistent source state at one unknown coordinate v . A fuzzy or multi-view image whose keys came from different uncoordinated views is outside

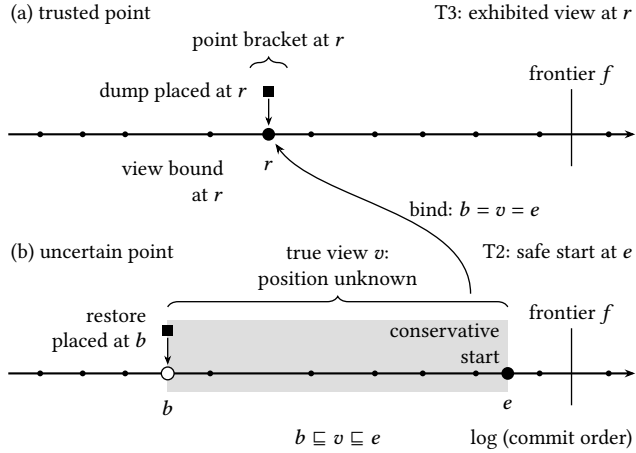


Figure 5: The degradation family. A trusted coordinate gives a point bracket with an exhibited shared-state anchor at r , which is T3 (Proposition 6). If the available recorded evidence establishes only $b \subseteq v \subseteq e$, the widened bracket is placed at T2 under the available recorded evidence, with its canonical trajectory guaranteed from the known coordinate e (Theorem 8). The trusted case is the degenerate bound $b = v = e = r$.

this placement unless a separate per-key mapping supplies the contract witnesses. Figure 5 draws both members of the family.

Theorem 8 (Degradation). Consider the dump plan of Proposition 6 with $A\text{-cert}$ and $A\text{-scope}$ dropped, so that no trusted binding establishes where the view v sits. Assume instead $A\text{-engine}$, $A\text{-engine-scope}$, and the uncertainty bound $b \subseteq v \subseteq e$, where the available recorded evidence bounds the true view between the backup’s start coordinate b and its post-recovery coordinate e , with $e \subseteq f$. The entire restored scope equals the source at that single v . Widen the unit’s bracket to (b, e) . Then:

- (1) the widened plan satisfies the contract, and Theorem 1 gives the cut at the frontier, reaching level T2. Because its only high edge is e , Lemma 4.2 also gives the canonical trajectory for every g with $e \subseteq g \subseteq f$.
- (2) the formal window-discard emission across the uncertainty window replays to the canonical sink at f . This statement requires an exact survivor listing, consumption beginning at or before b , observation of the scoped log in commit order through f , and closure at e only after every event through e has been processed. A finite physical emission achieves this final-replay equality when it starts from the empty state and realizes that construction with the stated survivor and observation conditions, by Corollary 4. The claim is equality after

replay through f , not prefix equivalence of the emitted stream (Theorem 2).

- (3) the trusted instance is the degenerate member of the same family. Proposition 6’s plan is the widened plan whose bound is already the point $b = v = e = r$, its coordinate trusted through $A\text{-cert}$ and $A\text{-scope}$.

PROOF. (1) O1 holds with the one widened unit. O2 holds with the unknown view v as each key’s existential witness. Here v lies in the bracket by the uncertainty bound, and validity at v is $A\text{-engine}$ with $A\text{-engine-scope}$. O3 and O5 are unchanged. The contract holds, Theorem 1 applies, and the latest high is e . (2) is Theorem 2 at this plan, with consumption from at or before b . (3) is immediate. Assumptions $A\text{-cert}$ and $A\text{-scope}$ give $v = p = r$, and Proposition 6’s point bracket at r is exactly the member of this family with $b = v = e = r$, the bound already a point. $\square$

What degrades is T3’s shared-state evidence. The latent view v still satisfies Corollary 1’s equations mathematically, but its operational exhibition is a meta-level fact rather than evidence available to the capture. Under the available recorded evidence, the interval member is therefore placed at T2 and receives the canonical trajectory from the known coordinate e , its high edge, by Lemma 4.2. The point member is T3 because its stated preconditions bind the read view to the recorded coordinate. Lacking that recorded exact binding adds deduplication work across the uncertainty window. It does not remove frontier equivalence or the post- e canonical trajectory. The emitted result remains final-only and additionally depends on the empty start, observation, close, and exact-survivor conditions.

Example 7.1 (The degradation bracket at work). A backup restores key a at value 4, read at some view between the backup’s start and its recovery point. After the view, the source updated a to 6 before the frontier. The uncertainty window spans both events. The discipline of Theorem 8(2) buffers the restored row $a \mapsto 4$, encounters the in-window update to 6, discards the buffer entry, and lets the event through. The replay yields 6, the source’s frontier value, with the stale restored value never surfacing. The untrusted coordinate cost this reconciliation pass over (b, e) , and nothing else.

Result. A native dump with a trusted point and the stated engine, scope, DDL, and representation conditions reaches T3. If the point is untrusted but the available recorded evidence bounds the true view inside an uncertainty interval, window-discard over that interval places the plan at T2 and supplies a canonical start at the interval’s high bound. If neither the point nor such a bound is justified, this placement does not apply.

8 Generalized DBLog at a Glance

This section is for practitioners and maintainers who operate, build, or extend DBLog, Debezium, Flink CDC, coordinate-bound capture, or native-dump workflows. It provides an operational entry to the framework without requiring the formal proofs. Proofs are contained in §7, while architectural boundaries, datastore assumptions, and out-of-scope features are cataloged in §12.

8.1 What Generalized DBLog is

Generalized DBLog is a correctness contract for interleaving copied database rows with an active change log. It covers the source database and capture side of a CDC pipeline, up to the emitted output stream. It is not a connector or an additional runtime layer. Instead, it defines what must be true for the output to reconstruct the source state without losing updates or resurrecting deleted data.

Under this contract, a connector implementation organizes its work around a declared scope, which is the set of row keys it intends to cover. The protocol divides that scope into copy units like chunks, tables, or key ranges. For each unit, it identifies the keys the unit owns, places the read between a low and high log position, and reconciles changes in that interval with the copied values. The unit closes only after the changes needed for reconciliation have been seen. An exact-position read is the special case where the low and high positions are the same. A whole dump can be one large unit, while parallel capture can use many non-overlapping units with different intervals. Figure 1 earlier in this paper shows how the original DBLog algorithm realizes this pattern with two watermark events around one chunk read.

When the contract holds, replaying the combined copy-and-log output reconstructs the exact state obtained by applying the source’s committed history up to one log position, key by key. We call that position the observation frontier, forming a virtual cut across the whole captured scope. The keys do not need to be read at the same time. Figure 6 makes this concrete for a four-key scope. At f , only the first unit has closed, so the reconstruction covers k_1 and k_2 while making no claim yet for the open unit (Lemma 4.3). At f' , both units have closed, and the four values at that cut match source-log replay at f' , even though no database snapshot was taken there. At f'' , the reconstruction also reflects a later committed change to k_3 . Once all units have closed, the log-wins reconstruction continues to match source-log replay at every later observed position.

The contract specifies correctness conditions, not operational mechanics. Whether a pipeline relies on write-based markers (classic DBLog), sampled GTID sets (Debezium read-only), parallel chunk readers (Flink CDC), or exact snapshot coordinates (MariaDB point-splice), they all fulfill the exact

same contract. Operational choices like chunk size, bracket width, write-free watermarking, and parallelism affect cost and throughput, but they do not weaken the guarantee when the core conditions hold.

A practitioner can assess a protocol by asking five questions. Which committed history is authoritative? What keys are in scope? Where does each read belong in that history? How are racing log events reconciled with the copy? Up to which log position is the reconstructed state guaranteed to match the source? The following two subsections divide the answers between the database and log on one side, and the connector implementation on the other. Together, the two lists cover the framework’s common source and capture requirements in operational language. The formal statements remain in Sections 2, 3, and 6. Individual protocol mappings may add requirements.

8.2 What the datastore and change log must provide

The framework requires several baseline guarantees from the database and its change log. If a database engine does not provide them natively, the connector must bridge the gap.

- (1) **A commit-ordered log.** Every committed change appears exactly once in the source history, in commit order. Events from active or aborted transactions do not appear. Only after a transaction commits does its complete ordered block of row changes enter this history.
- (2) **Log positions with a precise meaning.** A scalar offset, binlog position, LSN, or GTID set must identify exactly which prefix of the source history it represents. The chunk reads, bracket positions, and change-log events must all refer to that same history.
- (3) **A stable logical identity for each row.** The copy and log paths must map rows to identical logical keys. A primary-key update is modeled as deleting the old identity and inserting the new one. Tables lacking primary keys require another unique identifier. A sortable key is required only when partitioning tables via ordered chunk scans.
- (4) **Complete row images for logged changes.** Change events must carry full row after-images so newer changes can replace older state. If the log emits only deltas with modified columns, letting the log override the chunk copy would lose the unchanged columns, leaving an empty downstream sink with incomplete rows.

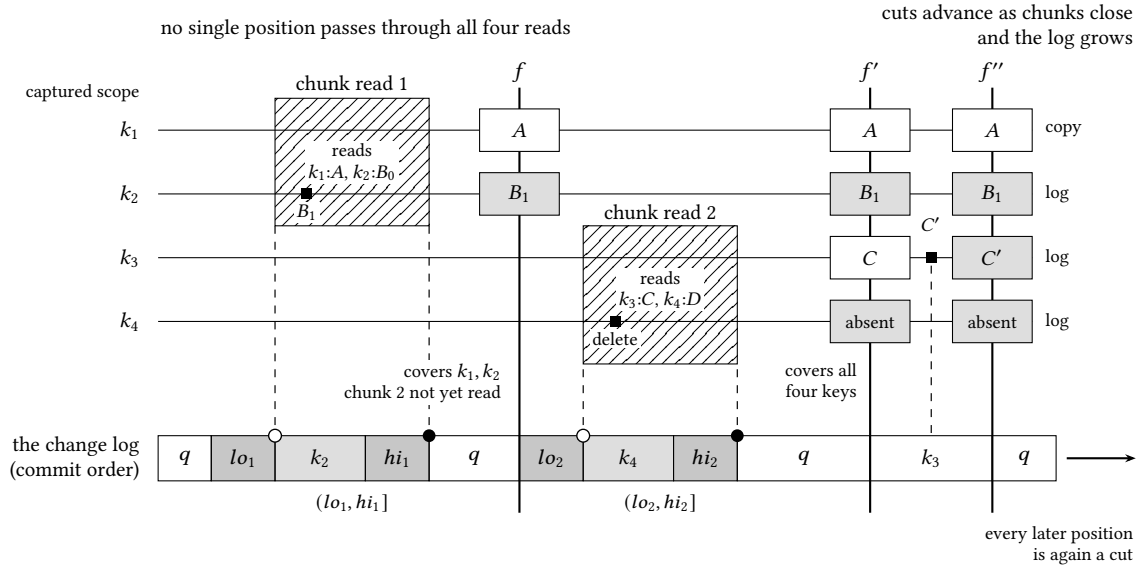


Figure 6: Virtual cuts on a four-key scope. Each copied value is valid somewhere between its unit’s low and high log positions. Its exact position is not observed and may differ by key. Reading down a cut shows the reconstructed state for each covered key, where white boxes indicate values retained from the row copy and gray boxes indicate values updated or deleted by the change log. In the log, q marks committed events for keys outside the captured scope.

- (5) **Committed, complete chunk reads.** The chunk scan must read committed data without silently skipping rows. Any row omitted by the scan must genuinely not exist at the time of the read. That row does not need to remain absent when the window closes, because a subsequent change-log insert will safely capture it.
- (6) **Sufficient log retention.** The database must retain its change log long enough for capture to process all events needed for reconciliation and continued replay. Continuous streaming connectors (like classic DBLog and Debezium) require the log to be retained from at or before each chunk’s low watermark, while parallel connectors (like Flink CDC) need past log segments to remain readable so workers can fetch and fold changes into chunks. If required log history is purged prematurely, capture cannot safely reconcile the copy. Related recovery and failover guidance is detailed in §8.7.

8.3 What the connector must establish

The connector implementation turns the source capabilities above into a complete handoff.

- (1) **A non-overlapping division of the scope.** The scope is the target set of keys to copy, whether a specific key range, a single table, or multiple tables.

Every key in this scope must belong to exactly one unit. Units may be chunks, ranges, or full tables, but their coverage must neither overlap nor leave gaps. The scope also includes keys that are absent initially and may be inserted while capture runs.

- (2) **Bracketed reads for every chunk.** Each chunk has a low and high log position. Each row key in the chunk’s range must reflect committed source state at some position inside that bracket. Different keys may match different coordinates. A single read is sufficient, but an implementation can also issue multiple reads within the bracket.
- (3) **Matching data types and operations.** Because chunk reads and change-log events arrive through different database interfaces, the connector must deserialize both into a canonical format and matching types. Operation meaning (inserts, updates, and deletes) must also be preserved during reconciliation rather than mapped into generic appends.
- (4) **The log must win over copied data.** Log events must always take precedence over older chunk reads. The reconciliation logic must ensure that stale copied rows never overwrite newer updates or resurrect deleted rows.
- (5) **No chunk closes before its window is fully processed.** A chunk cannot emit its rows until all change

events that raced the read have been reconciled. In streaming connectors (classic DBLog and Debezium), the connector tails the log starting at or before the low watermark, discarding buffered chunk rows as newer log events arrive, and releasing surviving rows only after consuming past the high watermark. In parallel connectors (Flink CDC), workers fetch the log segment between the low and high offsets from the server’s retained binlog to fold changes into the chunk. The live reader starts at or before every chunk’s high offset. It emits an event only when its position is strictly beyond the high offset of the event’s owning chunk.

- (6) **Final state equivalence of the output stream.** The output stream emitted by the connector does not need to match the historical change log event-for-event. However, replaying that emitted stream from an empty state must produce the exact same final state for every key as replaying the source log. This equivalence must hold across the entire handoff.

These conditions establish the copy-to-log guarantee described above. A production system that can restart or fail over has an additional lifecycle responsibility. It must recover the cursor, buffer, gate, and close state consistently, or fail closed and rebuild the incomplete unit. That guidance is discussed in §8.7.

8.4 How to read the observation frontier

The observation frontier is a single log position for the entire captured scope. The theorem guarantees that replaying the copy and log up to this frontier matches the source change log, key by key. However, because a single committed transaction may modify multiple rows, a log coordinate can land in the middle of that transaction’s row events.

For example, suppose keys a and b both start at 0, and a transaction commits changes setting both to 1. Its committed block of events is $[a \mapsto 1, b \mapsto 1]$. If replay stops after the first event, the reconstructed state is $a = 1, b = 0$. This state exposes no uncommitted data, but it is an intermediate step that never existed as an observable database state.

At any frontier, Generalized DBLog guarantees exact row-event replay. If the frontier also lands on a transaction boundary (known as *transaction closure*), the reconstructed state is also a consistent transactional snapshot. In short, reading committed data ensures clean input, but stopping at a transaction boundary is what guarantees a transactional snapshot. Protocols that select frontiers from transaction-complete coordinates (such as MySQL GTIDs) satisfy this condition naturally.

A connector that requires a transactional snapshot must ensure its frontier aligns with a transaction boundary. However, this does not guarantee atomic downstream application. A sink database that applies events individually may still expose intermediate states to its own readers. Downstream sink behavior remains outside this result.

8.5 How the protocol families fit

Table 1 shows how each protocol family establishes a bracket and reconciles copied rows with the change log. The framework’s correctness guarantees hold for a protocol when it satisfies both the baseline requirements from §§8.2-8.3 and the protocol-specific conditions described below. The table classifies these connector architectures rather than certifying individual releases.

The protocols rely on three reconciliation rules:

- (1) **Window-discard.** The connector buffers copied rows and forwards log events. An in-window event removes the buffered row with the same key. Only the surviving copied rows are emitted when the window closes.
- (2) **Range-merge.** The connector folds in-window log events directly into the copied rows, emits the reconciled chunk when the window closes, and streams subsequent log changes individually.
- (3) **Point splice.** The zero-width window case ($lo = hi$). The copy is placed at its exact log coordinate, and live log streaming resumes immediately from that point. When capturing multiple tables via point splices, log streaming must begin at or before the earliest recorded coordinate.

Under our baseline conditions, all three strategies produce the exact same final state as replaying the source log. Replay-ing the output stream matches the source database through the latest chunk’s high position and continues to match as subsequent log events arrive.

8.6 Conditions for the named protocols

Under the stated datastore and implementation conditions, the analyzed DBLog, Debezium, and Flink CDC protocol designs fit Generalized DBLog. These mappings identify the exact properties that a connector release and engine configuration must establish. The baseline datastore and connector requirements apply to every row in Table 1. The paragraphs below outline the specific handoff mechanisms and operational assumptions for each protocol.

Classic DBLog. The watermark table must be writable, both markers must be recognizable in the change log, and the chunk scan must cover its key range completely. The connector must consume past the high marker before emitting

Table 1: Generalized DBLog protocol mappings. The detailed conditions appear in §7.

Protocol shape	Bracket and scope	Reconciliation
Classic DBLog	Chunk between two captured marker events.	Window-discard.
Debezium signal-table (insert_insert, insert_delete)	Chunk bracketed by an opening signal-table event and either a closing insert or deletion of the opening row.	Window-discard, where a matching in-window event discards the buffered copy.
Debezium MySQL read-only	Chunk between exact complete executed-GTID prefix sets.	Window-discard by membership in high minus low.
Debezium MariaDB read-only	An empty difference gives a conditional point placement. A nonempty difference is placed here only for an empty buffer. Populated nonempty cases remain open.	Point splice at a confirmed quiet point. Otherwise window-discard with no copied survivors.
Flink CDC MySQL parallel chunks	Final logical chunks with low and high binlog offsets.	Range-merge each complete window, then gate the live suffix per split.
Coordinate-bound read or trusted dump	One scope, or multiple scopes sharing one exact read coordinate.	Copy-before-suffix point splice.
Bounded-uncertainty restore	The entire restored scope equals one source state at one unknown point inside justified bounds.	Window-discard over the whole interval.

surviving chunk rows. Theorem 6 proves that this original watermark protocol satisfies the generalized contract.

Debezium signal-table modes. In the default signaling mode (insert_insert), opening and closing records in a signaling table act as the low and high watermarks. The opening record must precede the chunk read, the closing record must follow it. The connector must not emit surviving rows until all change events within that window have been processed. Copied rows and change-log events must agree on key and value representations. Under insert_delete, the opening insert marks the low watermark and its subsequent deletion marks the high watermark. Debezium converts that deletion into an internal close signal, provided the delete event’s before-image retains the primary key [4,13,18].

Debezium MySQL. Debezium’s read-only MySQL mode uses Executed_Gtid_Set samples as chunk watermarks. Because MySQL documents non-atomic GTID externalization, each sampled GTID set must map to an exact, transaction-complete prefix of the change log [4,21]. In addition, the chunk read must cover its assigned key range, and surviving rows must not be emitted until the window reaches the closing GTID sample.

Debezium MariaDB. MariaDB’s GTID_BINLOG_POS records one frontier per domain, not MySQL’s complete executed set. Debezium 3.6.1.Final samples this vector and requests a reread when the high-minus-low GTID difference is nonempty. The helper performs the reread only when its snapshot, window-state, and buffer guards

hold [14,20,22,23]. Under the same-history and monotone-prefix assumptions, the conditional bridge accepts an empty difference as a confirmed quiet point only when the complete vector maps to the exact consumed prefix and covers all relevant domains. Under those assumptions, an empty difference gives a point splice for either a populated or empty chunk. With a nonempty difference, an empty buffer contributes no copied rows, and under the stated read, observation, and window-discard rules, any concurrent insert remains on the log path. These two analyzed cases use different proof structures but yield the same base guarantee (§7.4). These results do not cover a populated buffer with a nonempty GTID difference. Neither analyzed case uses the MySQL set-difference theorem or MySQL replica variables. Independently advancing domains still require an explicit same-history, prefix-preserving mapping.

For both read-only variants, the connector must record the low coordinate before starting the read and the high coordinate after the read completes. Surviving chunk rows may only be released after processing all row events belonging to the window’s final transaction. Out-of-scope events and server heartbeats needed to advance log coordinates must remain observable. Debezium’s PostgreSQL read-only incremental snapshot is not mapped here because its documentation does not specify explicit log brackets [11].

Flink CDC MySQL. Flink CDC partitions tables into chunk splits and uses range-merge reconciliation. The connector folds change-log events directly into each chunk split and emits the merged rows at the split’s high binlog offset. A live

Table 2: Operational lifecycle guidance outside the copy-to-log proof.

Discipline	Source history and recovery floor	Recoverable unit state and close condition	Restart or rebuild action	Safe purge and fail-closed rule
Window-discard	History from every open unit’s low edge and from the continuing reader.	Scope, low and high, survivor buffer, processed cursor, and whether every event through the high has closed the unit.	Resume from one consistent checkpoint, or discard the incomplete unit and reread it under a new bracket.	Purge only below every durable unit and reader floor. If required history is gone, do not release survivors. Rebuild the unit.
Range-merge	Complete low-to-high backfill for each unit and a live floor at or before every high.	Logical ownership, offsets, merged result, processed cursor, per-key gate, and completion of the high-edge merge.	Resume buffer and gate together, or discard the incomplete merged unit and reconstruct it from retained history.	Purge only when the merged result and gate durably represent the consumed range. If either cannot be recovered, fail closed and rebuild.
Point-splice	History from the splice coordinate for the continuing suffix.	Coordinate-bound scope, copy completion, suffix cursor, and handoff state.	Resume only while the same read view or completed copy remains valid. Otherwise obtain a new bound copy and splice point.	Purge below the splice only after the copy and suffix floor are durable. If the view or suffix is unavailable, do not claim the splice.

reader then streams subsequent change events. To prevent duplicate emissions, Flink uses a per-split gate that withholds live stream events whose positions fall at or before that split’s high offset. Chunk splits can be processed in any order, provided the split key is immutable. Updating a split column mid-capture can route rows across splits unpredictably.

Replay equivalence requires Flink’s changelog to preserve row operation metadata (RowKind for inserts, updates, and deletes). Append-only formats that discard operation types before reconciliation do not satisfy the contract. Backfill-skipping modes and tables lacking primary keys are not covered. Pipeline restarts, dynamic rescaling, failover, and sink-side exactly-once processing remain separate operational concerns.

Native dumps and physical restores. A point splice models native dump tools (such as mysqldump, pg_dump, and mariadb-dump) that bind an engine-consistent view to an exact recorded log coordinate. Because the window width is zero, no reconciliation is needed. Log consumption begins at or before that coordinate. In contrast, a bounded restore models physical backups and storage snapshots where the restored state is engine-consistent at an unknown coordinate within recorded bounds. The connector streams the log from the backup’s start, using window-discard to drop any restored rows modified before recovery completes. Replay is exact, and tracking the live source is guaranteed from the high bound onward.

8.7 Retention, restart, and failover guidance

Table 2 gives operational guidance for preserving the source and emission conditions across crashes, restarts, and database failovers. Its retry and rebuild actions must account

for prior output and every key affected by lost history, as required in §8.2. These are conditions for a recovery design, not a proved lifecycle protocol.

Safe crash recovery requires coordinating database log retention with connector checkpoints. The database must retain change logs back to the oldest position required by any open chunk or active reader. Purging logs ahead of that position prevents reconciliation, forcing the connector to fail closed and reread the affected units. When persisting connector progress, saved log offsets must match the state of in-memory chunk buffers and merge gates, using a transaction, write-ahead journal, or idempotent replay to survive worker restarts without drift.

Following a database failover, replication coordinates must map to the same continuous history or a verified, prefix-preserving continuation on the new primary. If coordinate continuity cannot be guaranteed, the connector must fail closed and rebuild the affected chunks rather than risk data corruption across divergent logs.

9 Making the 2020 Requirements Explicit

The 2020 DBLog paper listed two database requirements for capture, namely commit-ordered change events and non-stale reads [1]. While our framework validates the original algorithm under its stated assumptions, it also clarifies that these requirements were underspecified. On the read side, value freshness alone is not enough. The chunk scan must completely cover its assigned key range without silently skipping rows. If an unchanged row is missed by the scan, no change-log event will arrive to repair it. On the write side, watermarking implicitly requires table provisioning, write permissions, log coverage, recognizable marker events, and an ordered log. The generalized contract formalizes these

properties as explicit read obligations (O2 and O3, §3.3) and watermark preconditions (W1 through W5, §7.3), examined in Sections 9.1 and 9.2 below.

9.1 Read completeness

The 2020 paper states its chunk-read requirement twice, at different strengths. The strong form requires that “the selection executes on a specific position of the transaction log.” That means a single coordinate, a statement snapshot, which satisfies O2 with every key witnessed uniformly and is sufficient by Theorem 6. The text then generalizes, requiring that “the chunk selection sees the changes that are committed before its execution,” names this capability *non-stale reads*, and includes only this weaker condition in its Database Support section as the portability criterion [1]. The two statements are not equivalent, and this difference is precisely what Theorem 1 formalizes.

Per-row freshness already handles values that change during the scan. If a key changes inside the window, its log event removes or replaces the buffered row. If a key does not change, a value reflecting the source state at any coordinate in the bracket remains valid. This is why the theorem does not require all keys to share one read coordinate.

Scan completeness addresses a different requirement. Freshness constrains the values of rows that return, but says nothing about which rows return. Suppose a row remains present throughout the window and the scan silently skips it. Because the row never changed, the change log contains no event to supply it, and the connector advances to subsequent chunks without ever rescanning that range. The sink remains missing that row until some later write happens to update it. Even if the total refresh represents this omission as absence to satisfy O3 totality, O2 bracket validity still fails because the row remained present throughout the entire bracket. Together, O2 and O3 require every window-invariant present key to be returned with its valid value (§3.3).

The vendor documentation shows why this distinction matters. On MySQL and PostgreSQL, a read-committed select executes against a single statement snapshot [32,33]. The 2020 deployments therefore satisfied the stronger condition automatically. Under SQL Server’s default locking read committed, shared row locks are released as the scan advances. Microsoft’s locking guide documents that a concurrent key update can move a row so that it appears twice or is skipped entirely [34].

That documented case updates the row during the scan, meaning its log event would repair the omission under Case A of Theorem 1. While it is not a direct example of an unrepaired missing row, it establishes that lock-based scans can silently skip rows as concurrent updates move

them across pages. While we do not claim a live reproduction of data loss, the architectural conclusion remains clear: “non-stale reads” alone does not guarantee the scan completeness that DBLog relies on, whereas an engine statement snapshot guarantees it by design.

The complete condition is the contract’s combination of O2 with O3, with a statement snapshot serving as the canonical sufficient condition and the strong 2020 formulation as its uniform special case. Read against this, the 2020 text was correct for the engines it shipped on, but underspecified as a general portability criterion. The limitation lay in the portability criterion itself. An isolation-level label does not by itself establish complete per-key bracket validity, for which an engine statement snapshot provides one proven sufficient condition.

9.2 Watermark preconditions

The same Database Support section in the 2020 paper lists only commit-ordered change events and non-stale reads. The mechanism description, however, creates a dedicated watermark table and brackets every chunk with two updates whose events must appear in the change log [1]. Writability is therefore required by the mechanism but absent from the compatibility checklist. This matters for the non-relational stores the original text intended to cover, and also for read-only replicas. They can satisfy both listed conditions while being unable to run the watermark path.

Section 7.3 makes these requirements explicit. Conditions W1 through W5 state what the watermark mechanism actually requires, from table provisioning and write access to capture coverage, recognizability, and same-instance ordering coupling. None of them is new. Each is drawn directly from the 2020 paper’s own mechanism description. What is new is recognizing them as preconditions of one specific bracket construction rather than universal requirements of change-data-capture. Writability is a property of the chosen bracket mechanism, not of the datastore. The write-free instance of §7.4 satisfies the same contract obligations with no writes whatsoever, proving constructively that the 2020 watermark writes were simply one way to build brackets rather than an inherent requirement for correctness.

Both clarifications lead to the same conclusion. The original watermark mechanism remains sound, and the generalized contract cleanly separates that mechanism from the conditions needed to prove it correct. Section 7 then applies those same conditions to mechanisms that construct brackets without writes.

10 Mechanization

The normalized mathematical content of every numbered definition is represented, and every numbered result from

the cut theorem through the five placements and the degradation pair is proved in a machine-checked Isabelle/HOL development comprising about 5,000 lines of definitions and structured proofs, checked with Isabelle2025-2 over the standard library. Coordinate fidelity, O4, raw representation agreement under O5, and connector-release conformance are treated as instance evidence outside the prover. The development grounds the title’s “Verified” for the normalized theorem chain. The body’s mathematics stands alone. Each printed proof can be read against its mechanized counterpart or without it. The correspondence is organized by the paper’s own definition and theorem numbers.

The three verification layers have different scopes:

Isabelle/HOL. The full development covers the normalized mathematical form of every numbered definition and result, including the five placements and degradation. It contains about 5,000 lines and was checked by a clean session build. This is the paper’s general machine-checked proof.

Lean 4. The independent re-proof covers the contract core, cut theorem, the first three corollaries, window invariance, window-discard equivalence and monotonicity, and the no-shared-read-point witness over the core library. It is not a port of every placement or merge.

TLA⁺/TLC. Five models cover the classic, read-only, parallel, dump/degradation, and heterogeneous-composition protocols. Across 25 bounded runs they explore up to three keys, four events, and two chunks. The largest search has about 74 million distinct states. This is bounded evidence, not proof.

The classic instance imports the published formalization of the original algorithm through a session refactoring. Shared source and replay definitions are moved into a base session, and two identical virtual-cut predicates are consolidated. The wellformedness clauses and mathematical results are unchanged from the published development archived at [35]. Theorem 6 adopts those wellformedness clauses directly, and its source-state conclusion agrees with that development (§7.3).

The development is held to the submission standards of Isabelle’s Archive of Formal Proofs. It is axiom-free, with no statement left unproved and no proof step appealing to an external oracle. Definitions are conservative constructions, and the proofs are structured, human-readable Isar under a typeset proof document. The Isabelle session builds the complete chain, including the imported corpus, from a clean environment with permissive mode disabled. A clean build therefore checks every stated result rather than reusing a cached proof image.

The theorems are exercised as well as proved. Every countermodel this paper presents exists in the development as a constructed instance. The no-shared-read-point instance of Proposition 1, the no-early-onset composite of Proposition 2, and the window-narrowed variant of Proposition 3 each carry their failure proofs there, establishing no single shared read coordinate, no early onset, and a wrong replay, respectively. The degradation bracket of Example 7.1 is likewise mechanized as a concrete instance, and the positive theorems are instantiated at concrete plans in the same style rather than left abstract.

Where the effort went. The proof effort was not spread evenly across the development. The cut theorem itself was the cheap part. Its per-key argument is short in this paper and not much longer in the mechanization. Most of the effort went into the layers that keep it short. We kept the cut theorem free of observability, per-unit finiteness, and width assumptions, so those concerns had to be proved at the stream layer, where they apply. The two buffering disciplines account for roughly a third of the development’s lines, more than the contract and the cut theorem together. Most of that is in per-key emission-shape lemmas, duplicate-free survivor enumerations, and position tags. The countermodels were the other large expense. The theory that only constructs witnesses and their failure proofs is as long as the theory that holds the cut theorem. The remaining difficulty was discipline rather than proof. The classic formalization is imported as a separate session, so we matched its conventions exactly, with coordinate ties resolved by position and prefixes interpreted at-or-before, instead of adjusting them to fit. We consider the distribution itself informative. The mathematics was hardest where production systems differ, at the emissions and their equivalences, and not at the cut.

Model checking as a second check. Five TLA⁺ specifications model the classic watermarked loop, read-only executed sets, parallel range-merge, dump degradation, and a heterogeneous two-unit composite. They use the same half-open windows, per-key validity, and emission rules as the paper. Across 25 bounded runs, TLC checked frontier replay, per-key monotonicity for the chunked emissions, executed-set window membership, the trusted-splice and safe latest-high trajectories, degradation, and mixed composition. These checks are bounded. The general result comes from the machine-checked proof above.

Mutated runs recover the important failures. A forced shared read instant rejects Proposition 1’s per-key-witness construction. A window-only withholding rule produces Proposition 3’s wrong replay. Distinct splice coordinates refute an early common onset as in Proposition 2. A false trusted recovery point produces a stale splice, while a claim beginning inside the widened uncertainty bracket fails as

expected. These trajectory probes refute only the stronger earlier-onset claims. They do not refute the safe canonical trajectory beginning at the latest high edge. All contract runs append transactions atomically and check that no uncommitted event is visible. In one committed-only mutation, the current executed set is mapped to an interior prefix within an already committed transaction block. This breaks Theorem 7’s membership test without introducing an in-progress event. In a separate, explicitly out-of-contract probe, one transaction is artificially exposed in two pieces. The committed-only invariant fails immediately and the membership test can fail after a watermark is sampled in between. The boundary-valid window-discard replay survives both mutations on the small bounded searches, but this diagnostic robustness does not admit uncommitted input into the theory. TLC establishes bounded counterexamples when these assumptions are omitted, rather than a general theorem about replay. The models carry latent read positions as state, so TLC can evaluate the resulting equations. They do not model whether a running capture knows or can operationally exhibit such a position.

A second prover. The contract core, Theorem 1, Corollaries 1, 2, and 3, and the no-shared-read-point instance of Proposition 1, are also re-proved in Lean 4 over its core library alone, so that a second, independent proof kernel checks the central results.

Artifact availability. The accompanying artifact contains the Isabelle/HOL development and its imported dependencies, the Lean 4 formalization, and the five TLA⁺ models, together with build instructions and verification records [36]. The artifact’s Zenodo DOI is 10.5281/zenodo.22643866.

11 Related Work

ARIES provides a useful comparison from database recovery [37]. Its fuzzy checkpoints record transaction and dirty-page metadata while execution continues, without requiring dirty pages to be flushed. Restart uses page LSNs to guide redo and performs transaction undo where required. Generalized DBLog also reconciles state with ordered history, but works with committed row states and full post-images. O2 requires bracket-local validity without recording a version for each copied component. The comparison is therefore between reconciliation problems, not identical checkpoint or replay mechanisms.

Online schema-change tooling runs the same interleave inside a single database. gh-ost builds a ghost table, copies the original into it incrementally, and applies concurrent changes captured from the binary log onto the ghost before a final cut-over [38]. pt-online-schema-change copies in chunks while triggers on the original forward concurrent

modifications into the copy [39]. The anatomy of §3 is recognizably present, a copy, racing changes, reconciliation, and a cut-over, with the log as the reconciliation channel in one tool and triggers in the other. Both tools are described here, and neither is claimed. Whether either reconciliation fulfills the equivalence obligation of §3.4 is left open, and the trigger channel in particular is not a commit-ordered log, so even admissibility (§2.2) would need its own argument.

The splice shape recurs outside databases. Kubernetes’ list-then-watch protocol begins by listing a collection, takes the returned resourceVersion as its coordinate, and watches for changes after that version. When the server no longer covers that version it answers with 410 Gone, and the client discards its state and lists again [40]. The client lists, records a coordinate, resumes from the coordinate, and re-bootstraps when the log no longer reaches it. This is the point splice of §6.4 in an API-server setting, and the re-list rule handles a failure this paper treats as an observability failure (S-OBS). The shape, not membership, is what the citation records. Whether resourceVersion admits a faithful prefix reading is not examined here. Within databases, PostgreSQL’s logical replication bootstraps each table with its own tablesync worker before handoff to the shared apply stream [17]. Section 5.3 names the resulting composite after exactly this shape, and Proposition 2 gives its composition rule. Vitess VReplication copies key ranges and gates the event stream per copied range [41]. Its discipline is the key-range entry of §6.5, stated there as an open obligation.

In method, this paper belongs to a verification literature that rarely treats this subject. Machine-checked systems artifacts run from crash-safe storage, where specifications carry explicit crash conditions [42], to distributed-system implementations proved correct against refinement specifications [43]. Industrial practice, separately, checks designs by bounded exploration of formal specifications [44]. This paper does not verify an implementation. It verifies a contract and proves conditional placements for documented protocol designs. In this respect it is closer to specification frameworks that characterize a database guarantee by what clients can observe [45] than to verified implementations.

The nearest works are this paper’s own predecessors. The classic algorithm’s formalization [6] proves the certified virtual-cut object this framework incorporates at T4, and the state in Theorem 6 is precisely its replayed row-event state (§7.3). The delivery-side companion [16] studies recovery and sink acceptance after capture, outside the scope of this paper (§12). The 2020 paper [1] remains the family’s operational source, and §9 completes its requirements in the framework’s vocabulary rather than revisiting its design.

12 Scope and Limitations

The framework defines the capture contract up to the emitted event stream, proving that replaying this stream from an empty initial state reconstructs the source database. While the formal theorems assume an empty state, the emitted stream is also expected to work when updating an already populated sink. Copied rows and logged changes update the corresponding rows in the sink. However, a row that no longer exists at the source can remain in an already populated sink if no delete is sent for it. In practice, clearing the target scope beforehand or emitting explicit tombstones resolves this difference.

Extending the formal proofs to verify such populated-sink repairs, alongside transport, duplicate delivery, and exactly-once application, remains outside the current framework. Delivery is treated separately in [16]. Schema and DDL evolution are excluded by the stable logical identity space. The dump requirements in §7.7 reflect this limitation as a documented restriction on concurrent DDL.

Checkpoint, restart, rescaling, and failover protocols are not proved. The operational table in §8.7 is guidance about preserving protocol invariants, not a verified lifecycle result. A resumed run obtains the theorem’s guarantee only when its recovered plan and output again satisfy the same source and emission conditions.

The analysis is based on the 3.6 documentation for both projects and the source code of Debezium 3.6.1.Final and Flink CDC 3.6.0. These results evaluate protocol designs rather than certifying individual software releases. A connector realizes these guarantees whenever its runtime execution fulfills the modeled bracket and emission rules under the stated preconditions.

One common requirement across all protocols concerns scope discovery. Enumerating tables and key ranges from the database catalog is itself a read that must correspond to a valid log coordinate (§2.4). For native dumps, this enumeration is bound directly to the tool’s recorded server-wide snapshot position.

The paragraphs below catalog the specific operational assumptions and open evidence questions for each analyzed protocol.

Debezium signal-table modes. The conditional bridge covers the documented `insert_insert` and `insert_delete` strategies. It requires both window-edge events to enter the consumed history in the correct order around the chunk read, together with complete chunk read results (A-read). The connector must discard buffered rows matching in-window change events (A-discard), withhold surviving chunk rows until the closing signal is consumed from the log, and observe all intermediate log events without gaps (S-OBS). The

signaling collection must also satisfy the watermark preconditions (W1–W5, §7.3). Enabling the source signal channel does not itself prove that its collection is captured or its records recognizable. Under `insert_delete`, the deletion supplies the close only when its before-image retains the signal row’s identifier. No Debezium release is verified by this bridge [4,13,18].

Debezium MySQL. The direct set bridge requires every accepted `Executed_Gtid_Set` value to map to an exact gap-free prefix of complete transactions in the same history used by the chunk read and consumed events (A-linear, §7.4). Because MySQL explicitly documents non-atomic GTID externalization [21], the framework does not infer the needed semantic prefix property solely from when the status variable becomes visible. Complete chunk reads (A-read below), raw-to-projected GTID alignment, stable identity (A-scope), full row images (S-IMG), observation of state-advancing filtered events (S-OBS), and window-discard timing (A-discard) remain explicit conditions.

Debezium MariaDB. Unlike MySQL’s executed GTID set, MariaDB’s `GTID_BINLOG_POS` records a vector of per-domain frontiers. The conditional bridge maps this vector to a quiet point when its high-minus-low difference is empty across all domains, assuming the vector maps to a single consumed-history prefix. When that difference is nonempty, tagged 3.6.1.Final source code requests a reread. The shared helper requires the snapshot to be running, the deduplication window to be open, and the buffer to be populated before the reread occurs [20,23]. The code therefore does not establish that every populated attempt with a nonempty difference is retried. If the buffer is empty, there are no copied rows to release. Under the conditions in §7.4, processing the log still gives correct replay. The quiet-point and empty-buffer cases do not cover every path through the tagged implementation. A populated buffer with a nonempty difference requires further analysis of how the sampled coordinates bound the read and how the connector reconciles it with logged changes. This limits what the present analysis establishes about the connector.

Debezium PostgreSQL read-only (unmapped). Debezium’s PostgreSQL read-only incremental snapshot protocol is not mapped in this framework. Its documented watermark mechanism relies on in-progress transaction identifiers rather than commit-ordered WAL positions [11]. Because our theory reasons strictly about committed log history, in-progress transaction states lie outside the model. This boundary reflects the published connector documentation rather than a defect in the connector release.

Shared chunk-read condition. Official documentation for Debezium and Flink CDC does not fully specify the scan

completeness required by A-read (§7.4, §7.5). Because standard transaction isolation labels like `READ COMMITTED` do not inherently guarantee that a query will avoid skipping rows mid-scan, our mappings treat complete chunk reads as an explicit engine precondition rather than inferring it from an isolation-level name.

Flink CDC. While the abstract capture plan satisfies the contract independently, physical emitted-stream equivalence requires a correct streaming handoff. The connector must ensure full log retention across each chunk’s window (A-window), faithful merging of updates and deletes at the high watermark (A-merge), complete log observation (S-OBS), and per-split gating that withholds live stream events at or before the split’s high offset to prevent duplicates (A-handoff, §7.5). Chunk reads must additionally satisfy complete scanning (A-read). If a table is partitioned using a non-primary key column, that split key must remain immutable during capture to prevent rows from moving between splits. In addition, Flink’s default deserializer preserves insert, update, and delete semantics in `RowKind`, whereas append-only output formats erase operation types and are covered only through an explicit normalization proof [25]. Connector checkpointing, dynamic rescaling, failover, and end-to-end exactly-once sink application remain outside the formal scope. Modes that skip backfill reads or capture tables lacking primary keys are also omitted from our mappings.

Uncertain restores. The bounded restore theorem assumes that the restored dataset represents a single engine-consistent source state at some unknown coordinate within recorded log bounds. It does not cover inconsistent or torn file copies assembled from uncoordinated reads across different times. While an exact source state exists at some latent coordinate within the window, the connector cannot identify it from the recorded evidence alone. Consequently, tracking the live source is guaranteed only from the known high bound onward.

Session-bound snapshot coordinates. In MariaDB point-splice capture, the snapshot status variables (`Binlog_snapshot_file` and `Binlog_snapshot_position`) must be queried from the exact session holding the consistent snapshot before that view is closed or replaced [10,28].

Open reconciliation designs. Section 6.5 analyzes four additional reconciliation mechanisms, namely key-range gating, version-max, re-read normalization, and rewind-by-preimage. While their formal proof obligations are defined in our model, establishing their full equivalence theorems remains open.

13 Conclusion

Generalized DBLog provides a common basis for extending and combining capture methods. Its central requirement is that copied rows and logged changes reconstruct the selected data correctly once copying and reconciliation are complete. The paper makes explicit which properties must come from the source and which must be established by the connector, so a new way of obtaining a copy can be assessed against the same contract.

We prove that the original watermarked DBLog protocol, Debezium’s signal-table modes, Debezium’s MySQL read-only protocol, and Flink CDC’s parallel-chunk protocol fulfil this contract. For Debezium’s MariaDB read-only protocol, the proof covers the quiet-point and empty-buffer cases, and a populated buffer with a nonempty GTID difference remains open. The proofs state the source guarantees and connector behavior required by each. Native dumps tied to exact log positions and backups whose log position lies within known bounds, satisfy the same contract.

For a read-only variant, the implementation must establish how sampled source information identifies the changes that overlap its reads. For parallel capture, it must preserve non-overlapping key ownership and prevent earlier log events from reverting the reconciled data to an older state. A native dump must be paired with the right place to continue in the log, and its keys and values must agree with those decoded from logged changes. These checks identify what needs to be established when the capture mechanism changes. Chunk size and parallelism can be chosen according to source load, buffering, and retention limits while preserving those requirements.

The composition result also allows one capture to use different methods for different tables or key ranges. A native dump of one table can be combined with chunked reads of another. Each method establishes its own handoff to the same source history and their assigned keys remain non-overlapping. The combined output then preserves the correctness guarantee. This gives implementations room to choose a suitable copy method for each part of the database.

The machine-checked development provides reusable proofs for these extensions and combinations. Delivery, crash recovery, and downstream application need to be verified separately. For maintainers and connector authors, the contract provides a concrete basis for checking an existing capture path, changing how it operates, or adding a new one while preserving the correct treatment of copied rows, updates, and deletes.

AI assistance disclosure

The author used Claude Fable 5, ChatGPT 5.5 and 5.6 Sol, and GPT 6 Astra to assist with the theory, the Isabelle/HOL and Lean formalizations, the TLA⁺ models, and drafting the paper’s prose. The author edited all of the prose and takes responsibility for the paper’s content and conclusions.

References

- [1] Andreas Andreakis and Ioannis Papapanagiotou. DBLog: A watermark based change-data-capture framework, 2020. URL <https://arxiv.org/abs/2010.12597v1>. arXiv:2010.12597v1.
- [2] Andreas Andreakis and Ioannis Papapanagiotou. DBLog: A generic change-data-capture framework. Netflix Tech Blog, <https://netflixtechblog.com/dblog-a-generic-change-data-capture-framework-69351fb9099b>, December 2019. Accessed 8 September 2026.
- [3] Jiri Pechanec. Incremental snapshots in Debezium. Debezium Blog, <https://debezium.io/blog/2021/10/07/incremental-snapshots/>, October 2021. Published 7 October 2021. Accessed 8 September 2026.
- [4] Debezium Community. Debezium connector for MySQL. Debezium 3.6 reference documentation, <https://debezium.io/documentation/reference/3.6/connectors/mysql.html>, 2026. Accessed 8 September 2026.
- [5] Apache Flink CDC Community. MySQL CDC connector. Apache Flink CDC 3.6.0 documentation, <https://nightlies.apache.org/flink/flink-cdc-docs-release-3.6/docs/connectors/flink-sources/mysql-cdc/>, 2026. Accessed 8 September 2026.
- [6] Andreas Andreakis. A theoretical study of DBLog: Certified virtual cuts for a snapshot-equivalent replay of live databases, 2026. URL <https://arxiv.org/abs/2605.31475v4>. arXiv:2605.31475v4.
- [7] MariaDB. START TRANSACTION ... WITH CONSISTENT SNAPSHOT. MariaDB Server documentation, <https://mariadb.com/docs/server/ha-and-performance/standard-replication/enhancements-for-start-transaction-with-consistent-snapshot>, n.d. Accessed 8 September 2026.
- [8] Percona LLC. Start transaction with consistent snapshot. Percona Server for MySQL 8.0 documentation, <https://docs.percona.com/percona-server/8.0/start-transaction-with-consistent-snapshot.html>, 2025. Documentation revision dated November 27, 2025. Accessed 8 September 2026.
- [9] PostgreSQL Global Development Group. Streaming replication protocol. PostgreSQL 18 documentation, <https://www.postgresql.org/docs/18/protocol-replication.html#PROTOCOL-REPLICATION-CREATE-REPLICATION-SLOT>, 2026. Accessed 8 September 2026.
- [10] MariaDB. Replication and binary log status variables. MariaDB Server documentation, https://mariadb.com/docs/server/ha-and-performance/standard-replication/replication-and-binary-log-status-variables#binlog_snapshot_file, n.d. Accessed 8 September 2026.
- [11] Debezium Community. Debezium connector for PostgreSQL. Debezium 3.6 reference documentation, <https://debezium.io/documentation/reference/3.6/connectors/postgresql.html>, 2026. Accessed 8 September 2026.
- [12] Oracle Corporation. Monitoring Oracle GoldenGate processing: Using automatic heartbeat tables to monitor. Administering Oracle GoldenGate 19c (19.1.0), E98073-06, Section 16.4, <https://docs.oracle.com/en/middleware/goldengate/core/19.1/admin/monitoring-oracle-goldengate-processing.html#GUID-59E61274-BDDE-4D4B-9681-ED0BC39E9FCF>, 2024. Accessed 8 September 2026.
- [13] Debezium Community. Sending signals to a Debezium connector. Debezium 3.6 reference documentation, <https://debezium.io/documentation/reference/3.6/configuration/signalling.html>, 2026. Accessed 8 September 2026.
- [14] Debezium Community. Debezium connector for MariaDB. Debezium 3.6 reference documentation, <https://debezium.io/documentation/reference/3.6/connectors/mariadb.html>, 2026. Accessed 8 September 2026.
- [15] Oracle Corporation. mysqldump — a database backup program. MySQL 8.4 Reference Manual, <https://dev.mysql.com/doc/refman/8.4/en/mysqldump.html>, 2026. Accessed 8 September 2026.
- [16] Andreas Andreakis. Machine-checked dual-write recovery from a committed log, 2026. URL <https://arxiv.org/abs/2608.00501v4>. arXiv:2608.00501v4.
- [17] PostgreSQL Global Development Group. Logical replication architecture. PostgreSQL 18 documentation, <https://www.postgresql.org/docs/18/logical-replication-architecture.html#LOGICAL-REPLICATION-SNAPSHOT>, 2026. Accessed 8 September 2026.
- [18] Debezium Community. Signal dispatch and window-close sources for the insert_delete strategy. Debezium source at tag v3.6.1.Final, EventDispatcher, SignalRecord, and DeleteWindowCloser, 2026. Accessed 8 September 2026.
- [19] Kate Galieva. Read-only incremental snapshots for MySQL. Debezium Blog, <https://debezium.io/blog/2022/04/07/read-only-incremental-snapshots/>, April 2022. Published 7 April 2022. Accessed 14 September 2026.
- [20] Debezium Community. MySQL and binlog read-only incremental snapshot sources and common lifecycle. Debezium source at tag v3.6.1.Final, MySQL implementation, MySQL window context, shared binlog implementation, snapshot lifecycle, lifecycle context, 2026. Accessed 8 September 2026.
- [21] Oracle Corporation. GTID life cycle. MySQL 8.4 Reference Manual, <https://dev.mysql.com/doc/refman/8.4/en/replication-gtids-lifecycle.html>, 2026. Accessed 8 September 2026.
- [22] MariaDB. Global transaction ID. MariaDB Server documentation, https://mariadb.com/docs/server/ha-and-performance/standard-replication/gtid#gtid_binlog_pos, n.d. Accessed 8 September 2026.
- [23] Debezium Community. MariaDB read-only incremental snapshot source and context. Debezium source at tag v3.6.1.Final, MariaDB source and context, 2026. Accessed 8 September 2026.
- [24] MariaDB. Parallel replication. MariaDB Server documentation, <https://mariadb.com/docs/server/ha-and-performance/standard-replication/parallel-replication#out-of-order-parallel-replication>, n.d. Accessed 8 September 2026.
- [25] Apache Flink CDC Community. MySQL snapshot, backfill, handoff, and row-kind source paths. Apache Flink CDC source at tag release-3.6.0, snapshot task, snapshot buffer, record normalization, handoff, stream gate, row-kind mapping, append-only mapping, 2026. Accessed 8 September 2026.
- [26] MariaDB. MyRocks START TRANSACTION WITH CONSISTENT SNAPSHOT. MariaDB documentation, <https://mariadb.com/docs/server/server-usage/storage-engines/myrocks/myrocks-and-start-transaction-with-consistent-snapshot>, n.d. Accessed 8 September 2026.
- [27] PostgreSQL Global Development Group. Logical decoding concepts. PostgreSQL 18 documentation, <https://www.postgresql.org/docs/18/logicaldecoding-explanation.html#LOGICALDECODING-EXPLANATION-EXPORTED-SNAPSHOTS>, 2026. Accessed 8 September 2026.

- [28] MariaDB Server Contributors. MariaDB snapshot position capture, retrieval, and transaction reset. MariaDB Server source at tag mariadb-11.4.8, snapshot capture, status retrieval, and transaction reset, 2025. Source revision dated July 28, 2025. Accessed 8 September 2026.
- [29] Oracle Corporation. Using Oracle Flashback technology. Oracle Database Development Guide, 19c, E96334-10, Chapter 20, <https://docs.oracle.com/en/database/oracle/oracle-database/19/adfns/flashback.html>, 2026. Accessed 8 September 2026.
- [30] MariaDB. mariadb-dump. MariaDB documentation (rolling page, no version banner), <https://mariadb.com/docs/server/clients-and-utilites/backup-restore-and-import-clients/mariadb-dump>, n.d. Accessed 8 September 2026.
- [31] PostgreSQL Global Development Group. pg_dump. PostgreSQL 18 documentation, <https://www.postgresql.org/docs/18/app-pgdump.html>, 2026. Accessed 8 September 2026.
- [32] Oracle Corporation. Consistent nonlocking reads. MySQL 8.4 Reference Manual, <https://dev.mysql.com/doc/refman/8.4/en/innodb-consistent-read.html>, 2026. Accessed 8 September 2026.
- [33] PostgreSQL Global Development Group. Transaction isolation. PostgreSQL 18 documentation, <https://www.postgresql.org/docs/18/transaction-iso.html#XACT-READ-COMMITTED>, 2026. Accessed 8 September 2026.
- [34] Microsoft Corporation. Transaction locking and row versioning guide. SQL Server documentation, Microsoft Learn, <https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-ver17>, 2026. Accessed 8 September 2026.
- [35] Andreas Andreakis. Isabelle/HOL formal development for “A theoretical study of DBlog”, 2026. URL <https://doi.org/10.5281/zenodo.21732790>. Software, version 2.1, BSD 3-Clause license. Concept DOI 10.5281/zenodo.20389696.
- [36] Andreas Andreakis. Formal development for “Generalized DBlog: A verified contract for interleaving copied rows with a change log”, 2026. URL <https://doi.org/10.5281/zenodo.22643866>. Software, version 1.0, BSD 3-Clause license.
- [37] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. doi: 10.1145/128765.128770.
- [38] Shlomi Noach. gh-ost: GitHub’s online schema migration tool for MySQL. The GitHub Blog, <https://github.blog/news-insights/company-news/gh-ost-github-s-online-migration-tool-for-mysql/>, August 2016. Published 1 August 2016. Accessed 8 September 2026.
- [39] Daniel Nichter and Baron Schwartz. pt-online-schema-change. Percona Toolkit documentation, <https://docs.percona.com/percona-toolkit/pt-online-schema-change.html#description>, 2026. Percona Toolkit 3.7.1-4. Versioned documentation source. Accessed 8 September 2026.
- [40] The Kubernetes Authors. Kubernetes API concepts: Efficient detection of changes. <https://v1-36.docs.kubernetes.io/docs/reference/using-api/api-concepts/#efficient-detection-of-changes>, 2026. Kubernetes v1.36 documentation. Page updated March 30, 2026. Accessed 8 September 2026.
- [41] The Vitess Authors. VReplication: Life of a stream. Vitess 24.0 documentation, <https://vitess.io/docs/24.0/reference/vreplication/internal/life-of-a-stream/#copy>, 2025. Page updated October 29, 2025. Accessed 8 September 2026.
- [42] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M. Frans Kaashoek, and Nikolai Zeldovich. Using crash Hoare logic for certifying the FSCQ file system. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP)*, pages 18–37, 2015. doi: 10.1145/2815400.2815402.
- [43] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. Iron-Fleet: Proving practical distributed systems correct. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP)*, pages 1–17, 2015. doi: 10.1145/2815400.2815428.
- [44] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon Web Services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015. doi: 10.1145/2699417.
- [45] Natacha Crooks, Youer Pu, Lorenzo Alvisi, and Allen Clement. Seeing is believing: A client-centric specification of database isolation. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, pages 73–82, 2017. doi: 10.1145/3087801.3087802.